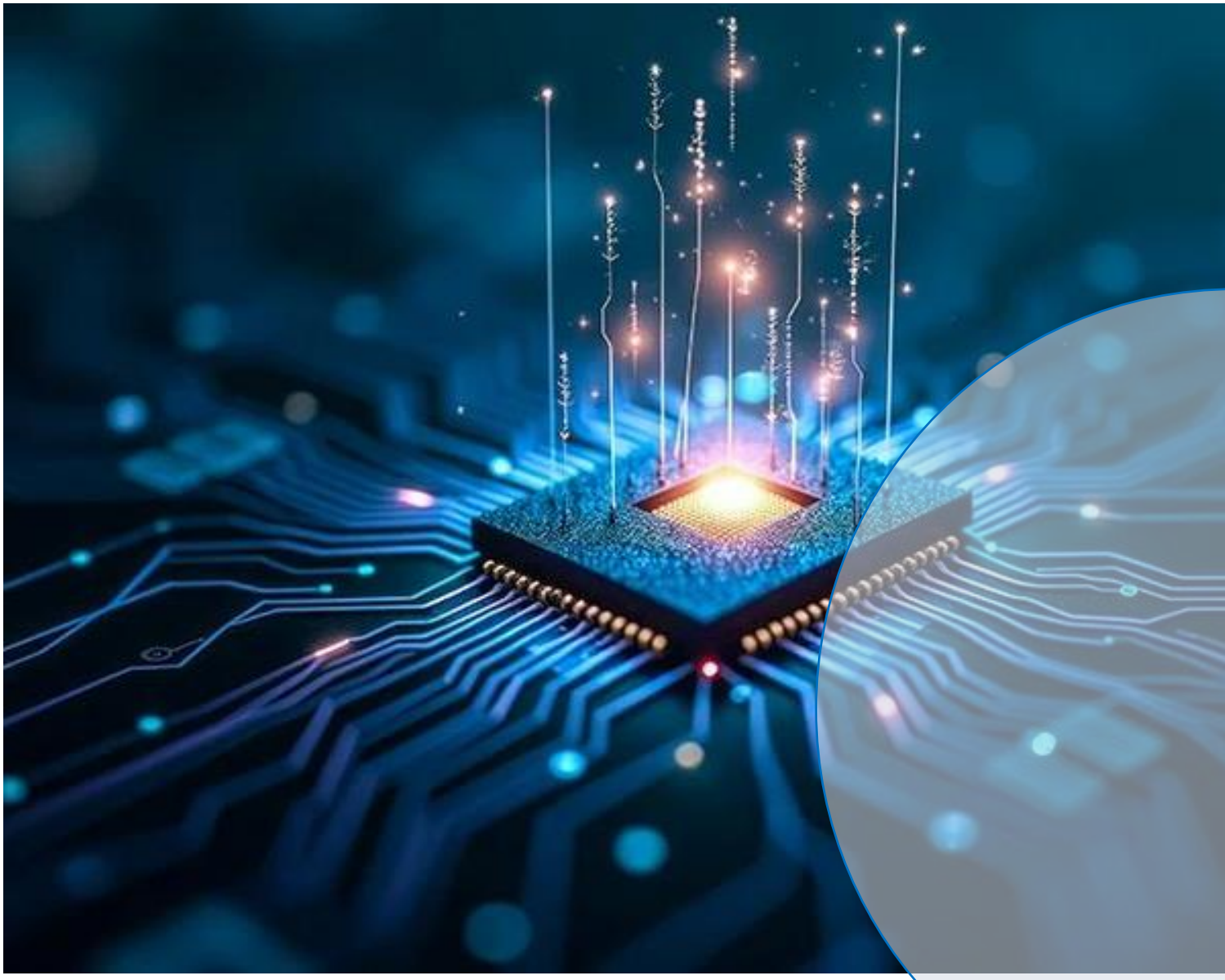


UVM Verification of a synchronous FIFO

Prepared by:

Haneen Fady Shahin

TABLE OF CONTENTS

Synchronous FIFO IP Core 2
- Executive Summary 2
- Technical Specifications & Parameterization 3
- Core Microarchitecture & Operational Logic 4
- Edge-Case Handling & Non-Destructive Protection 4

UVM_Testbench_Structure: 5

UVM_Testbench_Description: 6

Bugs_report: 11
- Before: 11
- After: 13

FIFO_UVM: 15
- Codes: 15
- Top: 15
- Design: 15
- Interface: 18
- Shared_pkg: 18
- Test: 19
- Env: 21
- Driver: 22
- Configuration_object: 23
- Sequence: 24
- Sequencer: 28
- Sequence_item: 29
- Agent: 30
- Monitor: 31
- Scoreboard: 32
- Functional_Coverage_collector: 36
- Assertions: 39

Verification_plan: 42

Src_files_list: 45

Src_dut_list: 45

Do_file: 45

Waveform: 46
- Rst_seq: 46
- Writing: 46
- Reading: 47
- Writing & reading: 47

Transcript: 49

Assertions: 50
- Assertions table: 50

Code_directives: 51

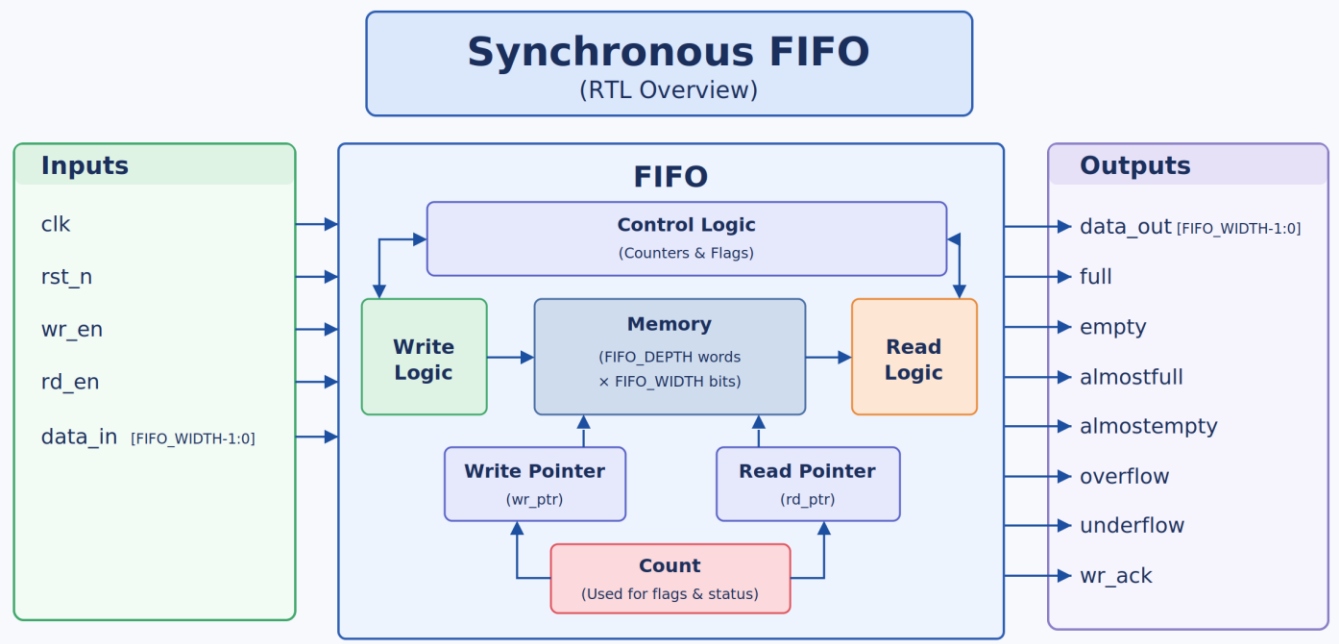
Coverage_groups: 51

Coverage_report: 53

Synchronous FIFO IP Core

Executive Summary

This report presents the functional architecture and operational behavior of a parameterizable **Synchronous First-In-First-Out (FIFO)** memory buffer IP core. The core serves as a fundamental building block in digital systems to safely buffer and pass data between logic modules operating on a single, shared clock domain. Operating on a strict First-In-First-Out queuing sequence, the module ensures that incoming data words are retrieved in the exact order they were written.



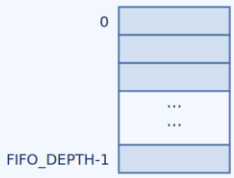
FIFO Operation

- Synchronous design (all operations on posedge clk)
- Write when wr_en = 1 and not full
- Read when rd_en = 1 and not empty
- Supports simultaneous read/write (special handling when both are 1)

Key Conditions

Condition	Action
wr_en = 1, full = 0	Write data, wr_ack = 1
wr_en = 1, full = 1	No write, overflow = 1
rd_en = 1, empty = 0	Read data
rd_en = 1, empty = 1	No read, underflow = 1
wr_en = 1, rd_en = 1	Both operations (special cases)

Memory Organization



Memory size:
FIFO_DEPTH × FIFO_WIDTH bits

Status Flags

- full : count == FIFO_DEPTH
- empty : count == 0
- almostfull : count == FIFO_DEPTH - 1
- almostempty : count == 1

Internal Logic (Summary)

- **Count** is updated based on wr_en and rd_en in each clock cycle.
- Pointers (wr_ptr, rd_ptr) wrap around at FIFO_DEPTH-1.
- **Flags** are generated from count.
- **Overflow/Underflow** are asserted on invalid operations.
- **wr_ack** is asserted when a write is accepted (not full).

Special Cases (wr_en & rd_en both 1)

- If FULL → count decreases by 1
- If EMPTY → count increases by 1
- If NORMAL → count unchanged

Depth: FIFO_DEPTH | Data Width: FIFO_WIDTH

Technical Specifications & Parameterization

The design is parameterizable to accommodate various memory storage requirements:

- **FIFO_WIDTH** *(Default: 16)*: Defines the bit-width of input data (data_in), output data (data_out), and internal memory words.
- **FIFO_DEPTH** *(Default: 8)*: Defines the total depth (number of memory words) of the FIFO storage array.

Interface Signal Description

- **clk (Input, Clock)**: The master system clock signal to which all internal FIFO operations are synchronized.
- **rst_n (Input, Control)**: The active-low asynchronous master reset signal used to initialize the FIFO pointers, counters, and registers.
- **wr_en (Input, Control)**: The Write Enable signal that requests a data write operation into the FIFO when asserted high.
- **rd_en (Input, Control)**: The Read Enable signal that requests a data read operation from the FIFO when asserted high.
- **data_in (Input Data Bus, FIFO_WIDTH bits)**: The input data bus used to drive incoming data words into the memory array during a write operation.
- **data_out (Output Data Bus, FIFO_WIDTH bits)**: The sequential output data bus that presents the data word retrieved from the FIFO during a read operation.
- **full (Output, Combinational Flag)**: Asserts high when the FIFO reaches maximum capacity (count == FIFO_DEPTH), indicating that further write requests will be ignored.
- **almostfull (Output, Combinational Flag)**: Asserts high when exactly one storage location remains available before full capacity (count == FIFO_DEPTH - 1).
- **empty (Output, Combinational Flag)**: Asserts high when the FIFO contains no valid data (count == 0), indicating that further read requests will be ignored.
- **almostempty (Output, Combinational Flag)**: Asserts high when exactly one data word remains in memory (count == 1).
- **wr_ack (Output, Sequential Status)**: Write Acknowledge signal that asserts high on the clock cycle following a successful write operation.
- **overflow (Output, Sequential Status)**: Asserts high on the following clock cycle when a write operation is attempted while the FIFO is full (full == 1).
- **underflow (Output, Sequential Status)**: Asserts high on the following clock cycle when a read operation is attempted while the FIFO is empty (empty == 1).

Core Microarchitecture & Operational Logic

1. Synchronous Control & Reset

All write, read, pointer update, and sequential status flag operations occur on the rising edge of clk. Asserting an active-low reset (rst_n = 0) asynchronously clears pointers, counter, and sequential status flags to zero.

2. Pointer Management & Memory Indexing

- **Write Pointer (wr_ptr)**: Indexes the memory location for incoming writes. Advances by 1 upon every valid, non-full write request.
- **Read Pointer (rd_ptr)**: Indexes the memory location for outgoing reads. Advances by 1 upon every valid, non-empty read request.
- **Wraparound Logic**: Both pointers index memory addresses from 0 to FIFO_DEPTH - 1 and automatically wrap back around to index 0 upon exceeding maximum depth.

3. Occupancy Tracking (count)

An internal counter (count) tracks the total number of valid data words stored:

- **Write Only**: Increments count by 1.
- **Read Only**: Decrements count by 1.
- **Concurrent Read & Write**: Maintains count at its current value during simultaneous valid read/write operations.

Edge-Case Handling & Non-Destructive Protection

The core contains built-in protective logic to prevent buffer corruption during invalid operation requests:

- **Overflow Protection**: If a write is attempted while full == 1, the write request is rejected, the current memory contents remain unchanged, and overflow is asserted.
- **Underflow Protection**: If a read is attempted while empty == 1, the read request is rejected, the output bus maintains its state, and underflow is asserted.
- **Simultaneous Operational Arbitration**:
    - *When Full (full == 1)*: A simultaneous write/read request rejects the write (triggering overflow), executes the read operation, and decrements count to free up storage.
    - *When Empty (empty == 1)*: A simultaneous write/read request rejects the read (triggering underflow), executes the write operation, and increments count.

UVM_TESTBENCH_STRUCTURE:

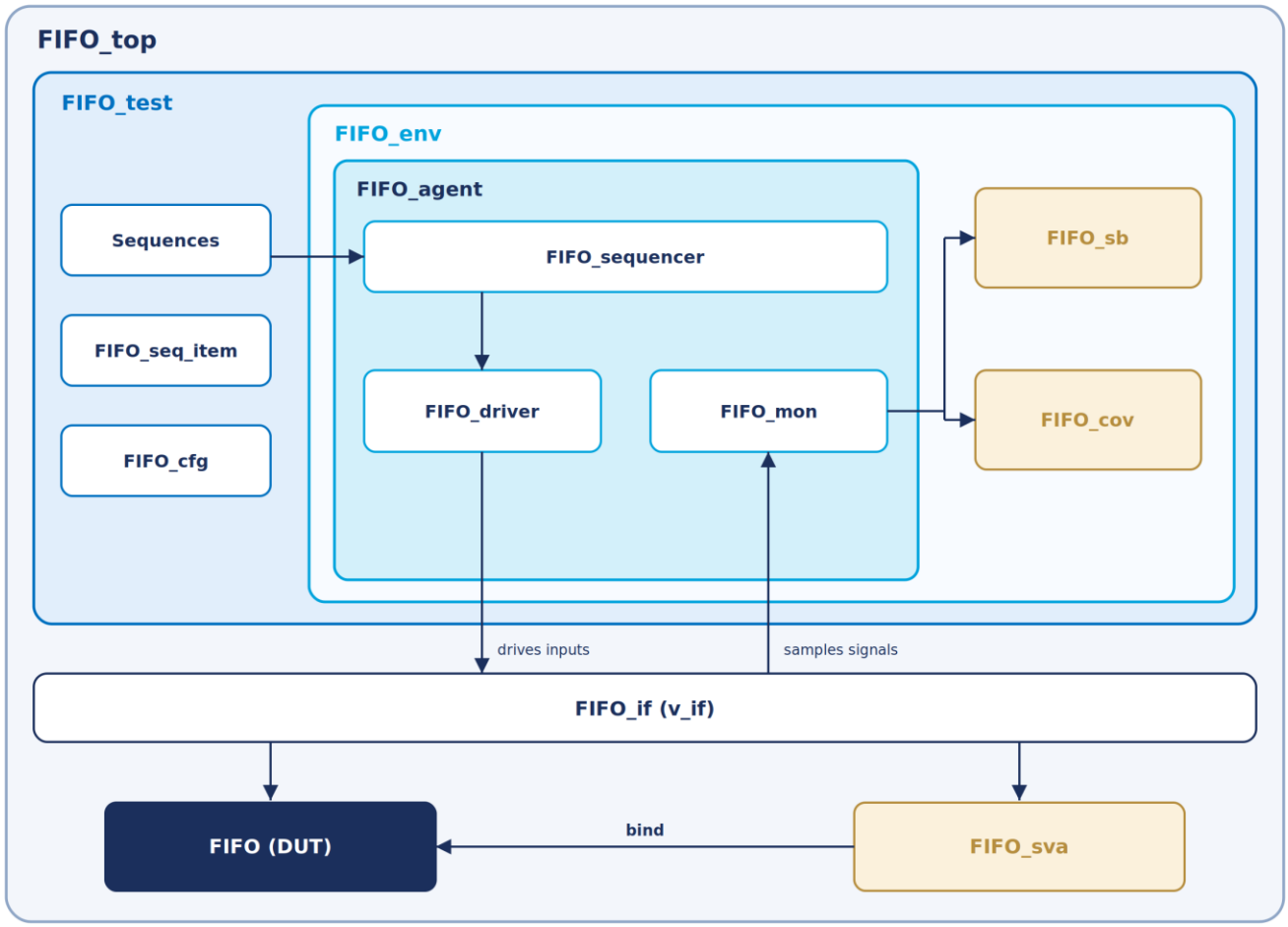


Figure: FIFO UVM testbench structure. Blue marks the test, cyan marks the agent, gold marks the checkers.

UVM_TESTBENCH_DESCRIPTION:

The FIFO is verified with a layered UVM testbench. A sequence creates transactions, the driver converts every transaction into pin activity on the FIFO interface, the monitor converts the pin activity back into a transaction, and the scoreboard, the coverage collector and the bound assertions analyze it. The sections below follow this order: top module, driving the interface, monitoring, and analyzing the output.

Component	Role
FIFO_top	Generates the clock, instantiates the interface and DUT, binds the assertions, stores the virtual interface and starts the test.
FIFO_if	Interface that groups all DUT pins plus the internal signals mem, wr_ptr, rd_ptr, count.
FIFO_test	Builds the environment and runs the sequences in order.
FIFO_config	Configuration object that carries the virtual interface handle.
FIFO_env	Builds and connects the agent, scoreboard and coverage collector.
FIFO_agent	Contains the sequencer, driver and monitor.
FIFO_seq_item	Transaction: stimulus fields, response fields and constraints.
FIFO_scoreboard	Golden reference model and output comparison.
FIFO_coverage	Functional coverage groups and crosses.
FIFO_sva	15 assertions (9 deferred immediate in `always_comb`, 6 concurrent) with cover directives, bound into the DUT.

1. Top module (FIFO_top)

FIFO_top is the only module of the testbench and drives no FIFO pin itself. It does five things:

- Generates the clock with forever #5 clk = ~clk, a 10 ns period.
- Instantiates the interface FIFO_if f_if(clk) and the DUT FIFO dut(f_if).
- Binds the assertion module into the DUT with bind FIFO FIFO_sva sva(f_if), so the assertions observe the same interface as the design.
- Stores the virtual interface in the UVM configuration database with uvm_config_db #(virtual FIFO_if)::set(null, "uvm_test_top", "FIFO_if", f_if).
- Calls run_test("FIFO_test"), which creates the test and starts the UVM phases.

2. Interface and configuration object

FIFO_if declares the parameters FIFO_WIDTH = 16 and FIFO_DEPTH = 8, the inputs (data_in, rst_n, wr_en, rd_en), the outputs (data_out, wr_ack, overflow, underflow, full, empty, almostfull, almostempty) and the internal signals mem, wr_ptr, rd_ptr and count, which the assertions

check. The clock is an input of the interface. UVM classes cannot connect to module ports, so they reach the pins through a virtual interface handle.

FIFO_config holds that handle (FIFO_vif). The test fetches the virtual interface from the configuration database, stores it in the configuration object and publishes the object with uvm_config_db #(FIFO_config)::set(this, "*", "CFG", cfg). The agent and the driver get it with the key CFG, and the agent hands the handle to the driver and the monitor in its connect_phase. A missing handle stops the run with a uvm_fatal.

3. Test and sequences (FIFO_test)

In build_phase the test creates the environment, the configuration object and the five sequences. In run_phase it raises an objection, starts the sequences one after the other on the agent sequencer (env.agt.sqr) and drops the objection when the last one ends. The order is:

1. FIFO_rst_seq: one item with rst_n = 0, data_in = 0, wr_en = 0, rd_en = 0. It puts the DUT and the reference model in a known reset state.
2. FIFO_wr_seq: 100 items with the write operation (wr_en = 1, rd_en = 0). It fills the FIFO and keeps writing after it is full.
3. FIFO_rd_seq: 100 items with the read operation (wr_en = 0, rd_en = 1). It drains the FIFO and keeps reading after it is empty.
4. FIFO_wr_rd_seq2: 100 items with wr_en = 1 and rd_en = 1 in every cycle.
5. FIFO_wr_rd_seq1: 100 repetitions of 10 write items followed by 10 read items (2000 items). The count goes up and down and both pointers wrap around.

Each sequence creates a FIFO_seq_item, calls start_item, sets wr_en and rd_en for its operation, calls randomize() and ends with finish_item. In total the test sends 2301 transactions.

4. Sequence item and constraints (FIFO_seq_item)

The item has the random stimulus fields data_in (16 bits), rst_n, wr_en and rd_en, and the response fields data_out, wr_ack, overflow, underflow, full, empty, almostfull and almostempty, which the monitor fills in. Three constraints shape the stimulus:

- rst_n dist {1 := 98, 0 := 2}: reset is asserted in about 2% of the items, so it occurs at random during every sequence.
- wr_en dist {1 := WR_EN_ON_DIST, 0 := 100 - WR_EN_ON_DIST} with WR_EN_ON_DIST = 70 from shared_pkg.
- rd_en dist {1 := RD_EN_ON_DIST, 0 := 100 - RD_EN_ON_DIST} with RD_EN_ON_DIST = 30 from shared_pkg.

data_in has no constraint, so every item carries a random 16-bit value. The item also provides convert2string and convert2string_stimulus for the log messages.

5. Driving the interface (sequencer, driver and agent)

The sequencer (FIFO_sequencer, a uvm_sequencer #(FIFO_seq_item)) passes the items of the running sequence to the driver one at a time. The agent creates the driver, sequencer and monitor in build_phase. In connect_phase it connects drv.seq_item_port to sqr.seq_item_export, gives the virtual interface to the driver and the monitor, and connects the monitor analysis port mon_ap to its own analysis port agt_ap.

The driver (FIFO_driver) gets the configuration object in build_phase and copies the virtual interface in connect_phase. Its run_phase is a forever loop:

1. get_next_item waits for the next item from the sequencer.
2. The driver writes data_in, rst_n, wr_en and rd_en of the item to the interface pins.
3. @(negedge FIFO_vif.clk) holds the inputs until the falling clock edge. The DUT samples them on the rising edge in between.
4. item_done tells the sequencer that the item is finished, and the next item is applied.

Inputs therefore change on the falling edge and are sampled by the DUT half a clock period later, so each item is applied for exactly one clock cycle.

6. Environment connections (FIFO_env)

The environment creates the agent (agt), the scoreboard (sb) and the coverage collector (cov). In connect_phase it connects the agent analysis port to both sb.sb_export and cov.cov_export, so every monitored transaction is broadcast to the scoreboard and the coverage collector.

7. Monitoring (FIFO_monitor)

The monitor only observes the interface and never drives it. In run_phase a forever loop creates a new item, waits for @(negedge FIFO_vif.clk), and copies the inputs (data_in, rst_n, wr_en, rd_en) and all outputs (data_out, wr_ack, overflow, underflow, full, empty, almostfull, almostempty) from the interface into the item. It then sends the item through mon_ap.write and prints it. One transaction therefore contains the inputs applied in a clock cycle together with the DUT outputs after that cycle. A new item object is created for every sample so the receivers can keep their own copy.

8. Analyzing the output: scoreboard (FIFO_scoreboard)

The scoreboard receives the items through sb_export into a uvm_tlm_analysis_fifo (sb_fifo). For each item it runs a golden reference model (ref_model) and compares the DUT outputs with the reference outputs. The model keeps its own memory (mem_ref), write pointer, read pointer and count, and works like this:

- **Reset** (rst_n = 0): pointers and count are cleared, and wr_ack_ref, overflow_ref and underflow_ref are set to 0.
- **Write only**: if the FIFO is not full, the data is stored, wr_ack_ref is set, the write pointer moves and the count increases. If it is full, overflow_ref is set and nothing changes.
- **Read only**: if the FIFO is not empty, data_out_ref takes the word at the read pointer, the read pointer moves and the count decreases. If it is empty, underflow_ref is set.
- **Write and read together**: when full only the read happens and overflow_ref is set; when empty only the write happens and underflow_ref is set; otherwise both happen and the count is unchanged.
- **Flags**: full_ref, empty_ref, almostfull_ref and almostempty_ref are computed from the count (FIFO_DEPTH, 0, FIFO_DEPTH - 1, 1).

The check uses !== on data_out, wr_ack, overflow, underflow, full, empty, almostfull and almostempty. A mismatch raises a uvm_error and increments error_count; a match increments correct_count. In report_phase the scoreboard prints the total successful and failed transactions. In this run all 2301 transactions were correct and none failed.

9. Functional coverage (FIFO_coverage)

The coverage collector receives the same items through cov_export and cov_fifo. For every item it samples cg_rst, and for items with rst_n = 1 it samples cg_FIFO.

- **cg_FIFO** coverpoints: wr_en, rd_en, full, almostfull, empty, almostempty, overflow, underflow and wr_ack.
- **Crosses**: cross_full, cross_almost_full, cross_empty and cross_almost_empty (each wr_en x rd_en x flag), and cross_overflow, cross_underflow and cross_wr_ack (each wr_en x rd_en x output). Impossible bins, such as overflow = 1 with wr_en = 0, are ignored with ignore_bins.
- **cg_rst**: the rst_n bins asserted, deasserted, assert_edge (1 to 0) and deassert_edge (0 to 1).

The flags full, empty, almostfull and almostempty in a monitored item are the values after the clock edge, while wr_en and rd_en were applied before it. The collector therefore keeps the flags of the previous item in the variables full_gate, empty_gate, almostfull_gate and almostempty_gate and crosses the inputs with those, which is the state the inputs were applied in. During reset the gates are set to the reset state (empty_gate = 1, the others 0). Both covergroups reached 100% (68 of 68 bins in cg_FIFO, 4 of 4 in cg_rst).

10. Assertions (FIFO_sva)

FIFO_sva is bound into the DUT in FIFO_top and is compiled when the SIM macro is defined. It contains 15 assertions, and each one has a cover directive. The checks that do not depend on a clock edge are written in one always_comb block as deferred immediate

assertions (assert final, with cover final), and the clocked behavior is checked with concurrent assertions (assert property, with cover property):

- **Asynchronous reset** (always_comb): when rst_n is low the outputs must already have their reset values, without waiting for a clock edge. Checks: reset_flags, RST_wr_ptr, RST_rd_ptr, RST_count, async_reset_seq.
- **Combinational flags** (always_comb): each flag must match the count. Checks: empty_flag, full_flag, almost_full, almost_empty.
- **Sequential outputs** (concurrent): write_ack, overflow_detection, underflow_detection.
- **Pointers and bounds** (concurrent): wr_wraparound, rd_wraparound, threshold.

assert final evaluates the expression at the final settled value of the time step, after the asynchronous reset and the continuous assignments have updated, so values that are only intermediate inside the time step are not reported. The assertions work next to the scoreboard: the assertions check reset, flag, pointer and counter behavior, and the scoreboard checks data and outputs against the golden model.

11. Flow of one transaction

1. The sequence randomizes an item and sends it with start_item and finish_item.
2. The sequencer gives the item to the driver through get_next_item.
3. The driver applies the item to FIFO_if and holds it until the falling clock edge.
4. The DUT samples the inputs on the rising clock edge and updates its outputs, and the bound assertions evaluate.
5. On the falling edge the monitor samples the inputs and outputs into a new item.
6. The monitor sends the item through mon_ap and agt_ap to the environment.
7. The scoreboard compares the item with the golden model, and the coverage collector samples the covergroups.
8. After the last sequence the test drops its objection, and report_phase prints the scoreboard summary.

Bugs_report:

1- wr_ack and overflow were absent in rst_n branch.

2- if condition(full & wr_en) used "&" not bitwise and "&&".

```
if (f_if.full && f_if.wr_en) //use " && " (bitwise and) instead of " & "----------> Warning: this statement is not a bitwise operation
    f_if.overflow <= 1;
```

3- underflow is sequential output and it was handled combinationally in assign statement.

4- almostfull was asserted when count equals (FIFO_DEPTH-2), It is fixed to (FIFO_DEPTH-1).

```
assign f_if.almostfull = (f_if.count == f_if.FIFO_DEPTH-1)? 1 : 0; //------------------->fixed from (count == f_if.FIFO_DEPTH-2) to (count == f_if.FIFO_DEPTH-1)
```

5- write and read operations weren't handled occurring at the same time, this branch is added.

Before:

```
////////////////////////////////////////////////////////////////////////////////
// Author: Kareem Waseem
// Course: Digital Verification using SV & UVM
//
// Description: FIFO Design
//
////////////////////////////////////////////////////////////////////////////////
module FIFO(data_in, wr_en, rd_en, clk, rst_n, full, empty, almostfull, almostempty, wr_ack, overflow,
underflow, data_out);
parameter FIFO_WIDTH = 16;
parameter FIFO_DEPTH = 8;
input [FIFO_WIDTH-1:0] data_in;
input clk, rst_n, wr_en, rd_en;
output reg [FIFO_WIDTH-1:0] data_out;
output reg wr_ack, overflow;
output full, empty, almostfull, almostempty, underflow;

localparam max_fifo_addr = $clog2(FIFO_DEPTH);

reg [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];

reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
reg [max_fifo_addr:0] count;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        wr_ptr <= 0;
```

```
    end
    else if (wr_en && count < FIFO_DEPTH) begin
        mem[wr_ptr] <= data_in;
        wr_ack <= 1;
        wr_ptr <= wr_ptr + 1;
    end
    else begin
        wr_ack <= 0;
        if (full & wr_en)
            overflow <= 1;
        else
            overflow <= 0;
    end
end

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        rd_ptr <= 0;
    end
    else if (rd_en && count != 0) begin
        data_out <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1;
    end
end

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        count <= 0;
    end
    else begin
        if  ( ({wr_en, rd_en} == 2'b10) && !full)
            count <= count + 1;
        else if ( ({wr_en, rd_en} == 2'b01) && !empty)
            count <= count - 1;
    end
end

assign full = (count == FIFO_DEPTH)? 1 : 0;
assign empty = (count == 0)? 1 : 0;
assign underflow = (empty && rd_en)? 1 : 0;
assign almostfull = (count == FIFO_DEPTH-2)? 1 : 0;
assign almostempty = (count == 1)? 1 : 0;

endmodule
```

After:

```
////////////////////////////////////////////////////////////////////////////////
// Author: Kareem Waseem
// Course: Digital Verification using SV & UVM
//
// Description: FIFO Design
//
////////////////////////////////////////////////////////////////////////////////
module FIFO(FIFO_if.dut f_if);

localparam max_fifo_addr = $clog2(f_if.FIFO_DEPTH);

reg [f_if.FIFO_WIDTH-1:0] mem [f_if.FIFO_DEPTH-1:0];

reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
reg [max_fifo_addr:0] count;

always_ff @(posedge f_if.clk or negedge f_if.rst_n) begin
    if (!f_if.rst_n) begin
        wr_ptr <= 0;
        f_if.wr_ack <= 0; //---------------------------->added
        f_if.overflow <= 0; //-------------------------->added
    end
    else if (f_if.wr_en && count < f_if.FIFO_DEPTH) begin
        mem[wr_ptr] <= f_if.data_in;
        f_if.wr_ack <= 1;
        wr_ptr <= wr_ptr + 1;
        f_if.overflow<=0;
    end
    else begin
        f_if.wr_ack <= 0;
        if (f_if.full && f_if.wr_en) //use " && " (bitwise and) instead of " & "----------> Warning: this
statement is not a bitwise operation
            f_if.overflow <= 1;
        else
            f_if.overflow <= 0;
    end
end

always_ff @(posedge f_if.clk or negedge f_if.rst_n) begin
    if (!f_if.rst_n) begin
        rd_ptr <= 0;
        f_if.underflow<=0; //------------------->fixed (handling sequentially)
    end
    else if (f_if.rd_en && count != 0) begin
        f_if.data_out <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1;
```

```
        f_if.underflow<=0; //---------------------->fixed (handling sequentially)
    end
    else begin
        if (f_if.empty && f_if.rd_en )begin
            f_if.underflow<=1; //--------------------->fixed (handling sequentially)
        end
        else begin
            f_if.underflow<=0; //--------------------->fixed (handling sequentially)
        end
    end
end

always_ff @(posedge f_if.clk or negedge f_if.rst_n) begin
    if (!f_if.rst_n) begin
        count <= 0;
    end
    else begin
        //write
        if  ( ({f_if.wr_en, f_if.rd_en} == 2'b10) && !f_if.full)
            count <= count + 1;
        //read
        else if ( ({f_if.wr_en, f_if.rd_en} == 2'b01) && !f_if.empty)
            count <= count - 1;
        //handling write and read together
        else if ({f_if.wr_en, f_if.rd_en} == 2'b11) begin //------------------------------------->added
            if(f_if.full)begin
                count <= count - 1;
            end
            else if (f_if.empty) begin
                count <= count + 1;
            end
        end
    end
end

assign f_if.full = (count == f_if.FIFO_DEPTH)? 1 : 0;
assign f_if.empty = (count == 0)? 1 : 0;
//assign underflow = (f_if.empty && f_if.rd_en)? 1 : 0;   //------------------------------->fixed
(handling sequentially)
assign f_if.almostfull = (count == f_if.FIFO_DEPTH-1)? 1 : 0; //--------------------->fixed from (count ==
f_if.FIFO_DEPTH-2) to (count == f_if.FIFO_DEPTH-1)
assign f_if.almostempty = (count == 1)? 1 : 0;
```

FIFO_UVM:

Codes:

➢ Top:

```
import uvm_pkg::*;
import shared_pkg::*;
import FIFO_test_pkg::*;
`include "uvm_macros.svh"

module FIFO_top();
    bit clk;
    initial begin
        clk=0;
        forever#5 clk=~clk;
    end

    FIFO_if f_if(clk);
    FIFO dut(f_if);

    bind FIFO FIFO_sva sva(f_if);

    // run test using run_test task
    initial begin
        uvm_config_db #(virtual FIFO_if)::set(null,"uvm_test_top","FIFO_if",f_if);
        run_test("FIFO_test");
    end


endmodule
```

➢ Design:

```
module FIFO(FIFO_if f_if);

always_ff @(posedge f_if.clk or negedge f_if.rst_n) begin
    if (!f_if.rst_n) begin
        f_if.wr_ptr <= 0;
        f_if.wr_ack <= 0; //---------------------------->added
        f_if.overflow <= 0; //-------------------------->added
    end
    else if (f_if.wr_en && f_if.count < f_if.FIFO_DEPTH) begin
        f_if.mem[f_if.wr_ptr] <= f_if.data_in;
        f_if.wr_ack <= 1;
        f_if.wr_ptr <= f_if.wr_ptr + 1;
        f_if.overflow<=0;
    end
```

```
    else begin
        f_if.wr_ack <= 0;
        if (f_if.full && f_if.wr_en) //use " && " (bitwise and) instead of " & "----------->
Warning: this statement is not a bitwise operation
            f_if.overflow <= 1;
        else
            f_if.overflow <= 0;
    end
end

always_ff @(posedge f_if.clk or negedge f_if.rst_n) begin
    if (!f_if.rst_n) begin
        f_if.rd_ptr <= 0;
        f_if.underflow<=0; //------------------------------->fixed (handling sequentially)
    end
    else if (f_if.rd_en && f_if.count != 0) begin
        f_if.data_out <= f_if.mem[f_if.rd_ptr];
        f_if.rd_ptr <= f_if.rd_ptr + 1;
        f_if.underflow<=0; //------------------------------->fixed (handling sequentially)
    end
    else begin
        if (f_if.empty && f_if.rd_en )begin
            f_if.underflow<=1; //------------------------------->fixed (handling
sequentially)
        end
        else begin
            f_if.underflow<=0; //------------------------------->fixed (handling
sequentially)
        end
    end
end

always_ff @(posedge f_if.clk or negedge f_if.rst_n) begin
    if (!f_if.rst_n) begin
        f_if.count <= 0;
    end
    else begin
        //write
        if  ( ({f_if.wr_en, f_if.rd_en} == 2'b10) && !f_if.full)
            f_if.count <= f_if.count + 1;
        //read
        else if ( ({f_if.wr_en, f_if.rd_en} == 2'b01) && !f_if.empty)
            f_if.count <= f_if.count - 1;
        //handling write and read together
        else if ({f_if.wr_en, f_if.rd_en} == 2'b11) begin //-------------------------------
---->added
            if(f_if.full)begin
                f_if.count <= f_if.count - 1;
```

```
            end
            else if (f_if.empty) begin
                f_if.count <= f_if.count + 1;
            end
        end
    end
end

assign f_if.full = (f_if.count == f_if.FIFO_DEPTH)? 1 : 0;
assign f_if.empty = (f_if.count == 0)? 1 : 0;
//assign underflow = (f_if.empty && f_if.rd_en)? 1 : 0;   //-------------------------------
>fixed (handling sequentially)
assign f_if.almostfull = (f_if.count == f_if.FIFO_DEPTH-1)? 1 : 0; //--------------------
>fixed from (count == f_if.FIFO_DEPTH-2) to (count == f_if.FIFO_DEPTH-1)
assign f_if.almostempty = (f_if.count == 1)? 1 : 0;

endmodule
```

➢ Interface:

```
interface FIFO_if(input bit clk);

    parameter FIFO_WIDTH = 16;
    parameter FIFO_DEPTH = 8;
    localparam max_fifo_addr = $clog2(f_if.FIFO_DEPTH);

    logic [FIFO_WIDTH-1:0] data_in;
    logic rst_n, wr_en, rd_en;

    logic [FIFO_WIDTH-1:0] data_out;
    logic  wr_ack, overflow;
    logic full, empty, almostfull, almostempty, underflow;

    //internal monitoring signals
    reg [f_if.FIFO_WIDTH-1:0] mem [f_if.FIFO_DEPTH-1:0];
    reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
    reg [max_fifo_addr:0] count;

endinterface
```

➢ Shared_pkg:

```
package shared_pkg;
    parameter FIFO_WIDTH = 16;
    parameter FIFO_DEPTH = 8;
    localparam max_fifo_addr = $clog2(FIFO_DEPTH);
    int RD_EN_ON_DIST=30;
    int WR_EN_ON_DIST=70;

endpackage:shared_pkg
```

➢ Test:

```
package FIFO_test_pkg;
  import uvm_pkg::*;
  import FIFO_env_pkg::*;
  import FIFO_config_pkg::*;
  import FIFO_rst_seq_pkg::*;
  import FIFO_wr_seq_pkg::*;
  import FIFO_rd_seq_pkg::*;
  import FIFO_wr_rd_seq_pkg::*;
 `include "uvm_macros.svh"


  class FIFO_test extends uvm_test;
    `uvm_component_utils(FIFO_test)
    FIFO_env env;
    FIFO_config cfg;
    virtual FIFO_if FIFO_vif;
    FIFO_rst_seq rst_seq;
    FIFO_wr_seq wr_seq;
    FIFO_rd_seq rd_seq;
    FIFO_wr_rd_seq1 wr_rd_seq1;
    FIFO_wr_rd_seq2 wr_rd_seq2;

    function new (string name="FIFO_test",uvm_component parent=null);
      super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
      super.build_phase(phase);
      env=FIFO_env::type_id::create("env",this);
      cfg=FIFO_config::type_id::create("cfg");
      rst_seq=FIFO_rst_seq::type_id::create("rst_seq");
      wr_seq=FIFO_wr_seq::type_id::create("wr_seq");
      rd_seq=FIFO_rd_seq::type_id::create("rd_seq");
      wr_rd_seq1=FIFO_wr_rd_seq1::type_id::create("wr_rd_seq1");
      wr_rd_seq2=FIFO_wr_rd_seq2::type_id::create("wr_rd_seq2");

      if(!(uvm_config_db #(virtual FIFO_if)::get(this,"","FIFO_if",cfg.FIFO_vif)))begin
        `uvm_fatal("build_phase","TEST-Unable to get virtual interface of the FIFO from
uvm_config_db")
      end

      uvm_config_db #(FIFO_config)::set(this,"*","CFG",cfg);
    endfunction

    task run_phase(uvm_phase phase);
      super.run_phase(phase);
```

```
      phase.raise_objection(this);
      //reset sequence
      `uvm_info("run_phase","Reset Asserted",UVM_LOW);
      rst_seq.start(env.agt.sqr);
      `uvm_info("run_phase","Reset Deasserted",UVM_LOW);
      //main sequence
      `uvm_info("run_phase","Stimulus Generation Started",UVM_LOW);
      //write
      `uvm_info("run_phase","Writing Started",UVM_LOW);
      wr_seq.start(env.agt.sqr);
      //read
      `uvm_info("run_phase","Reading Started",UVM_LOW);
      rd_seq.start(env.agt.sqr);
      //write and read simultaneously
      `uvm_info("run_phase","Writing and Reading simultaneously Started",UVM_LOW);
      wr_rd_seq2.start(env.agt.sqr);
      //write and read
      `uvm_info("run_phase","Writing and Reading Started",UVM_LOW);
      wr_rd_seq1.start(env.agt.sqr);
      //end stimulus
      `uvm_info("run_phase","Stimulus Generation Ended",UVM_LOW);
      phase.drop_objection(this);
    endtask

  endclass: FIFO_test
endpackage
```

➢ Env:

```
package FIFO_env_pkg;
import uvm_pkg::*;
import FIFO_agent_pkg::*;
import FIFO_scoreboard_pkg::*;
import FIFO_coverage_pkg::*;


`include "uvm_macros.svh"

class FIFO_env extends uvm_env;
  `uvm_component_utils(FIFO_env)
  FIFO_agent agt;
  FIFO_scoreboard sb;
  FIFO_coverage cov;


  function new(string name = "FIFO_env", uvm_component parent = null);
    super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    agt = FIFO_agent::type_id::create("agt",this);
    sb = FIFO_scoreboard::type_id::create("sb",this);
    cov = FIFO_coverage::type_id::create("cov",this);
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    agt.agt_ap.connect(sb.sb_export);
    agt.agt_ap.connect(cov.cov_export);
  endfunction
endclass
endpackage
```

➢ Driver:

```
package FIFO_driver_pkg;
    import uvm_pkg::*;
    import FIFO_seq_item_pkg::*;
    import FIFO_config_pkg::*;

    `include "uvm_macros.svh"

    class FIFO_driver extends uvm_driver #(FIFO_seq_item);
        `uvm_component_utils(FIFO_driver)

        virtual FIFO_if FIFO_vif;
        FIFO_seq_item stim_seq_item;
        FIFO_config cfg;

        function new(string name="FIFO_driver",uvm_component parent=null);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            if(!(uvm_config_db #(FIFO_config)::get(this,"","CFG",cfg)))begin
                `uvm_fatal("build_phase","Driver-Unable to get configuration object")
            end
        endfunction

        function void connect_phase(uvm_phase phase);
            super.connect_phase(phase);
            FIFO_vif=cfg.FIFO_vif;
        endfunction

        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            stim_seq_item = FIFO_seq_item::type_id::create("stim_seq_item");
            forever begin
                seq_item_port.get_next_item(stim_seq_item);
                FIFO_vif.data_in =stim_seq_item.data_in;
                FIFO_vif.rst_n =stim_seq_item.rst_n;
                FIFO_vif.wr_en =stim_seq_item.wr_en;
                FIFO_vif.rd_en =stim_seq_item.rd_en;
                @(negedge FIFO_vif.clk);
                seq_item_port.item_done();
                `uvm_info("run_phase",stim_seq_item.convert2string_stimulus(),UVM_HIGH)
            end
        endtask

    endclass:FIFO_driver
```

```
endpackage
```

➢ Configuration_object:

```
package FIFO_config_pkg;
    import uvm_pkg::*;
    `include "uvm_macros.svh"

    class FIFO_config extends uvm_object;
        `uvm_object_utils(FIFO_config)

        virtual FIFO_if FIFO_vif;

        function new(string name = "FIFO_config");
            super.new(name);
        endfunction

    endclass:FIFO_config
endpackage
```

➢ Sequence:

RST_seq:

```
package FIFO_rst_seq_pkg;
    import shared_pkg::*;
    import FIFO_seq_item_pkg::*;
    import uvm_pkg::*;
    `include "uvm_macros.svh"

    class FIFO_rst_seq extends uvm_sequence #(FIFO_seq_item);
        `uvm_object_utils(FIFO_rst_seq)
        FIFO_seq_item seq_item;

        function new(string name="FIFO_rst_seq");
            super.new(name);
        endfunction

        task body;
            seq_item=FIFO_seq_item::type_id::create("seq_item");
            start_item(seq_item);
            seq_item.rst_n=0;
            seq_item.data_in=0;
            seq_item.rd_en=0;
            seq_item.wr_en=0;
            finish_item(seq_item);
        endtask

    endclass:FIFO_rst_seq
endpackage
```

Write_seq:

```
package FIFO_wr_seq_pkg;
    import uvm_pkg::*;
    import shared_pkg::*;
    import FIFO_seq_item_pkg::*;
    `include "uvm_macros.svh"

    class FIFO_wr_seq extends uvm_sequence #(FIFO_seq_item);
        `uvm_object_utils(FIFO_wr_seq)
        FIFO_seq_item seq_item;

        function new(string name="FIFO_wr_seq");
            super.new(name);
        endfunction

        task body;
            seq_item=FIFO_seq_item::type_id::create("seq_item");
            repeat(100)begin
                start_item(seq_item);
                seq_item.wr_en=1;
                seq_item.rd_en=0;
                assert(seq_item.randomize());
                finish_item(seq_item);
            end
        endtask
    endclass
endpackage
```

Read_seq:

```
package FIFO_rd_seq_pkg;
    import uvm_pkg::*;
    import shared_pkg::*;
    import FIFO_seq_item_pkg::*;
    `include "uvm_macros.svh"

    class FIFO_rd_seq extends uvm_sequence #(FIFO_seq_item);
        `uvm_object_utils(FIFO_rd_seq)
        FIFO_seq_item seq_item;

        function new(string name="FIFO_rd_seq");
            super.new(name);
        endfunction

        task body;
            seq_item=FIFO_seq_item::type_id::create("seq_item");
            repeat(100)begin
                start_item(seq_item);
                seq_item.wr_en=0;
                seq_item.rd_en=1;
                assert(seq_item.randomize());
                finish_item(seq_item);
            end
        endtask
    endclass
endpackage
```

Write & read_seq:

```
package FIFO_wr_rd_seq_pkg;
    import uvm_pkg::*;
    import shared_pkg::*;
    import FIFO_seq_item_pkg::*;
    `include "uvm_macros.svh"

    class FIFO_wr_rd_seq1 extends uvm_sequence #(FIFO_seq_item);
        `uvm_object_utils(FIFO_wr_rd_seq1)
        FIFO_seq_item seq_item;

        function new(string name="FIFO_wr_rd_seq1");
            super.new(name);
        endfunction

        task body;
            seq_item=FIFO_seq_item::type_id::create("seq_item");
                repeat(100)begin
                    repeat(10)begin
                        start_item(seq_item);
                        seq_item.wr_en=1;
                        seq_item.rd_en=0;
                        assert(seq_item.randomize());
                        finish_item(seq_item);
                    end
                    repeat(10)begin
                        start_item(seq_item);
                        seq_item.wr_en=0;
                        seq_item.rd_en=1;
                        assert(seq_item.randomize());
                        finish_item(seq_item);
                    end
                end
        endtask
    endclass:FIFO_wr_rd_seq1

    class FIFO_wr_rd_seq2 extends uvm_sequence #(FIFO_seq_item);
        `uvm_object_utils(FIFO_wr_rd_seq2)
        FIFO_seq_item seq_item;

        function new(string name="FIFO_wr_rd_seq2");
            super.new(name);
        endfunction

        task body;
            seq_item=FIFO_seq_item::type_id::create("seq_item");
            repeat(100)begin
```

```
                start_item(seq_item);
                seq_item.wr_en=1;
                seq_item.rd_en=1;
                assert(seq_item.randomize());
                finish_item(seq_item);
            end

        endtask
    endclass:FIFO_wr_rd_seq2
endpackage
```

➢ Sequencer:

```
package FIFO_sequencer_pkg;
    import uvm_pkg::*;
    import FIFO_seq_item_pkg::*;
    `include "uvm_macros.svh"

    class FIFO_sequencer extends uvm_sequencer #(FIFO_seq_item);
        `uvm_component_utils(FIFO_sequencer)

        function new(string name="FIFO_sequencer",uvm_component parent=null);
            super.new(name,parent);
        endfunction

    endclass:FIFO_sequencer
endpackage
```

➢ Sequence_item:

```
package FIFO_seq_item_pkg;
    import uvm_pkg::*;
    import shared_pkg::*;
    `include "uvm_macros.svh"

    class FIFO_seq_item extends uvm_sequence_item;
        `uvm_object_utils(FIFO_seq_item)

        rand logic [FIFO_WIDTH-1:0] data_in;
        rand logic rst_n, wr_en, rd_en;

        logic [FIFO_WIDTH-1:0] data_out;
        logic  wr_ack, overflow;
        logic full, empty, almostfull, almostempty, underflow;

        function new(string name="FIFO_seq_item");
            super.new(name);
        endfunction

        function string convert2string();
            return $sformatf("%s rst_n=0b%0b data_in=0b%0b wr_en=0b%0b rd_en=0b%0b
data_out=0b%0b wr_ack=0b%0b overflow=0b%0b full=0b%0b empty=0b%0b almostfull=0b%0b
almostempty=0b%0b
underflow=0b%0b",super.convert2string(),rst_n,data_in,wr_en,rd_en,data_out,wr_ack,overflow,ful
l,empty,almostfull,almostempty,underflow);
        endfunction

        function string convert2string_stimulus();
           return $sformatf("rst_n=0b%0b data_in=0b%0b wr_en=0b%0b rd_en=0b%0b
",rst_n,data_in,wr_en,rd_en);
        endfunction


        //----------constraints------------//

        constraint rst_n_c{
            rst_n dist {1:=98,0:=2};
        }

        constraint wr_en_c {
            wr_en dist{1:=WR_EN_ON_DIST,0:=100-WR_EN_ON_DIST};
        }

        constraint rd_en_c {
            rd_en dist{1:=RD_EN_ON_DIST,0:=100-RD_EN_ON_DIST};
        }
```

```
    endclass:FIFO_seq_item
endpackage
```

➢ Agent:

```
package FIFO_agent_pkg;
    import uvm_pkg::*;
    import FIFO_seq_item_pkg::*;
    import FIFO_config_pkg::*;
    import FIFO_driver_pkg::*;
    import FIFO_sequencer_pkg::*;
    import FIFO_monitor_pkg::*;
    `include "uvm_macros.svh"

    class FIFO_agent extends uvm_agent;
        `uvm_component_utils(FIFO_agent)

        FIFO_driver drv;
        FIFO_sequencer sqr;
        FIFO_monitor mon;
        FIFO_config cfg;
        //analysis port
        uvm_analysis_port #(FIFO_seq_item) agt_ap;

        function new(string name = "FIFO_agent",uvm_component parent=null);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            if(!(uvm_config_db #(FIFO_config)::get(this,"","CFG",cfg)))begin
                `uvm_fatal("build_phase","Unable to get configuration object")
            end

            //create drv,sqr,mon
            drv = FIFO_driver::type_id::create("drv",this);
            sqr = FIFO_sequencer::type_id::create("sqr",this);
            mon = FIFO_monitor::type_id::create("mon",this);
            agt_ap = new("agt_ap",this);
        endfunction

        function void connect_phase(uvm_phase phase);
            super.connect_phase(phase);
            drv.FIFO_vif=cfg.FIFO_vif;
            mon.FIFO_vif=cfg.FIFO_vif;
            drv.seq_item_port.connect(sqr.seq_item_export);
            mon.mon_ap.connect(agt_ap);
```

```
        endfunction

    endclass:FIFO_agent
endpackage
```

➢ Monitor:

```
package FIFO_monitor_pkg;
    import uvm_pkg::*;
    import shared_pkg::*;
    import FIFO_seq_item_pkg::*;
    `include "uvm_macros.svh"

    class FIFO_monitor extends uvm_monitor;
        `uvm_component_utils(FIFO_monitor)

        uvm_analysis_port #(FIFO_seq_item) mon_ap;
        virtual FIFO_if FIFO_vif;
        FIFO_seq_item rsp_seq_item;

        function new(string name="FIFO_monitor",uvm_component parent=null);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            mon_ap =new("mon_ap",this);
        endfunction

        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            forever begin
                rsp_seq_item=FIFO_seq_item::type_id::create("rsp_seq_item");
                @(negedge FIFO_vif.clk);
                rsp_seq_item.data_in =FIFO_vif.data_in;
                rsp_seq_item.rst_n =FIFO_vif.rst_n;
                rsp_seq_item.wr_en =FIFO_vif.wr_en;
                rsp_seq_item.rd_en =FIFO_vif.rd_en;
                rsp_seq_item.data_out =FIFO_vif.data_out;
                rsp_seq_item.wr_ack =FIFO_vif.wr_ack;
                rsp_seq_item.full =FIFO_vif.full;
                rsp_seq_item.empty =FIFO_vif.empty;
                rsp_seq_item.almostfull =FIFO_vif.almostfull;
                rsp_seq_item.almostempty =FIFO_vif.almostempty;
                rsp_seq_item.overflow =FIFO_vif.overflow;
                rsp_seq_item.underflow =FIFO_vif.underflow;
                mon_ap.write(rsp_seq_item);
```

```
                `uvm_info("run_phase",rsp_seq_item.convert2string(),UVM_HIGH)
            end
        endtask
    endclass
endpackage
```

➢ Scoreboard:

```
package FIFO_scoreboard_pkg;
    import uvm_pkg::*;
    import shared_pkg::*;
    import FIFO_seq_item_pkg::*;
    import FIFO_config_pkg::*;
    `include "uvm_macros.svh"

    class FIFO_scoreboard extends uvm_scoreboard;
        `uvm_component_utils(FIFO_scoreboard)

    uvm_analysis_export #(FIFO_seq_item) sb_export;
    uvm_tlm_analysis_fifo #(FIFO_seq_item) sb_fifo;
    FIFO_seq_item seq_item_sb;

    logic [FIFO_WIDTH-1:0] mem_ref [FIFO_DEPTH-1:0];
    logic [FIFO_WIDTH-1:0] data_out_ref;
    logic wr_ack_ref, overflow_ref;
    logic full_ref, empty_ref, almostfull_ref, almostempty_ref, underflow_ref;

    int error_count=0;
    int correct_count=0;

    function new(string name = "FIFO_scoreboard",uvm_component parent =null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        sb_export=new("sb_export",this);
        sb_fifo=new("sb_fifo",this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        sb_export.connect(sb_fifo.analysis_export);
    endfunction

    task run_phase(uvm_phase phase);
```

```
        super.run_phase(phase);
        forever begin
            sb_fifo.get(seq_item_sb);
            ref_model(seq_item_sb);
            if(seq_item_sb.data_out !== data_out_ref || seq_item_sb.wr_ack !== wr_ack_ref ||
seq_item_sb.overflow !== overflow_ref || seq_item_sb.full !== full_ref || seq_item_sb.empty
!== empty_ref || seq_item_sb.almostfull !== almostfull_ref || seq_item_sb.almostempty !==
almostempty_ref || seq_item_sb.underflow !== underflow_ref)begin
                `uvm_error("run_phase",$sformatf("Comparison failed, Transaction recieved by
the DUT:%s While the reference out:0b%0b" ,
seq_item_sb.convert2string(),data_out_ref,wr_ack_ref,overflow_ref,full_ref,empty_ref,almostful
l_ref,almostempty_ref,underflow_ref));
                error_count++;
            end
            else begin
                `uvm_info("run_phase",$sformatf("Correct
data_out:%s",seq_item_sb.convert2string()),UVM_HIGH);
                correct_count++;
            end
        end
    endtask


    function void ref_model(FIFO_seq_item FIFO_seq_item_chk);
        static logic [max_fifo_addr-1:0] wr_ptr;
        static logic [max_fifo_addr-1:0] rd_ptr;
        static bit [max_fifo_addr:0] count;
        bit read;

        if(!FIFO_seq_item_chk.rst_n)begin
            wr_ptr=0;
            rd_ptr=0;
            count=0;
            wr_ack_ref=0;
            overflow_ref=0;
            underflow_ref=0;
            read=0;
        end
        else begin
            wr_ack_ref=0;
            overflow_ref=0;
            underflow_ref=0;
            //handling write and read operations together
            if(FIFO_seq_item_chk.wr_en && FIFO_seq_item_chk.rd_en)begin
                //read only
                if(full_ref)begin
                    data_out_ref = mem_ref[rd_ptr];
                    rd_ptr = (rd_ptr + 1)% FIFO_DEPTH;
```

```
                count = count - 1;
                overflow_ref = 1;
                read=1;
            end
            //write only
            else if(empty_ref)begin
                mem_ref[wr_ptr] = FIFO_seq_item_chk.data_in;
                wr_ack_ref = 1;
                wr_ptr = (wr_ptr + 1)% FIFO_DEPTH;
                count = count + 1;
                underflow_ref=1;
                read=0;

            end
            else begin
                mem_ref[wr_ptr] = FIFO_seq_item_chk.data_in;
                wr_ack_ref = 1;
                wr_ptr = (wr_ptr + 1)% FIFO_DEPTH;
                data_out_ref = mem_ref[rd_ptr];
                rd_ptr = (rd_ptr + 1)% FIFO_DEPTH;
                read=1;
            end
        end
        //write
        else if (FIFO_seq_item_chk.wr_en) begin
            read = 0;
            if(!full_ref)begin
                mem_ref[wr_ptr] = FIFO_seq_item_chk.data_in;
                wr_ack_ref = 1;
                wr_ptr = (wr_ptr + 1)% FIFO_DEPTH;
                count = count + 1;
            end
            else begin
                overflow_ref=1;
            end
        end
        //read
        else if (FIFO_seq_item_chk.rd_en) begin
            if(!empty_ref)begin
                data_out_ref = mem_ref[rd_ptr];
                rd_ptr = (rd_ptr + 1)% FIFO_DEPTH;
                count = count - 1;
                read=1;
            end
            else begin
                underflow_ref=1;
                read=0;
            end
```

```
            end
            else begin
                read=0;
            end
        end

        full_ref        = (count == FIFO_DEPTH);
        empty_ref       = (count == 0);
        almostfull_ref  = (count == FIFO_DEPTH - 1);
        almostempty_ref = (count == 1);
    endfunction

    function void report_phase(uvm_phase phase);
        super.report_phase(phase);
        `uvm_info("report_phase",$sformatf("Total successful
transactions:%0d",correct_count),UVM_MEDIUM);
        `uvm_info("report_phase",$sformatf("Total failed
transactions:%0d",error_count),UVM_MEDIUM);
    endfunction

    endclass
endpackage
```

➢ Functional_Coverage_collector:

```
package FIFO_coverage_pkg;
    import uvm_pkg::*;
    import shared_pkg::*;
    import FIFO_seq_item_pkg::*;
    import FIFO_config_pkg::*;
    `include "uvm_macros.svh"

    class FIFO_coverage extends uvm_component;
        `uvm_component_utils(FIFO_coverage)
        uvm_analysis_export #(FIFO_seq_item) cov_export;
        uvm_tlm_analysis_fifo #(FIFO_seq_item) cov_fifo;
        FIFO_seq_item seq_item_cov;

        bit full_gate, empty_gate, almostfull_gate, almostempty_gate;
        //  Covergroup: cg_wr_rd
        //
        covergroup cg_FIFO ;

            //coverpoints

            c_wr_en:       coverpoint seq_item_cov.wr_en;
            c_rd_en:       coverpoint seq_item_cov.rd_en;
            c_full:        coverpoint full_gate;
            c_almostfull:  coverpoint almostfull_gate;
            c_empty:       coverpoint empty_gate;
            c_almostempty: coverpoint almostempty_gate;
            c_overflow:    coverpoint seq_item_cov.overflow;
            c_underflow:   coverpoint seq_item_cov.underflow;
            c_wr_ack:      coverpoint seq_item_cov.wr_ack;

            // full/empty/almostfull/almostempty now correlate correctly with
            // wr_en/rd_en, so every bin is genuinely reachable -- no ignore_bins needed
            cross_full:         cross c_wr_en,c_rd_en,c_full;
            cross_almost_full:  cross c_wr_en,c_rd_en,c_almostfull;
            cross_empty:        cross c_wr_en,c_rd_en,c_empty;
            cross_almost_empty: cross c_wr_en,c_rd_en,c_almostempty;

            // overflow/underflow/wr_ack are registered outputs computed FROM
            // the pre-edge count in the same cycle as wr_en/rd_en, so they
            // were already correctly aligned -- no gating needed here
            cross_overflow: cross c_wr_en,c_rd_en,c_overflow{
                ignore_bins no_wr_overflow = binsof(c_wr_en) intersect {0} &&
binsof(c_overflow) intersect {1};
            }
            cross_underflow: cross c_wr_en,c_rd_en,c_underflow{
```

```
                ignore_bins no_rd_underflow = binsof(c_rd_en) intersect {0} &&
binsof(c_underflow) intersect {1};
            }
            cross_wr_ack: cross c_wr_en,c_rd_en,c_wr_ack{
                ignore_bins no_wr_ack = binsof(c_wr_en) intersect {0} && binsof(c_wr_ack)
intersect {1};
            }

        endgroup: cg_FIFO

        covergroup cg_rst;
            c_rst_n: coverpoint seq_item_cov.rst_n {
                bins asserted      = {0};
                bins deasserted    = {1};
                bins assert_edge   = (1 => 0);
                bins deassert_edge = (0 => 1);
            }
        endgroup: cg_rst

        function new(string name = "FIFO_coverage",uvm_component parent = null);
            super.new(name,parent);
            cg_FIFO = new();
            cg_rst = new();
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            cov_export=new("cov_export",this);
            cov_fifo=new("cov_fifo",this);
        endfunction

        function void connect_phase(uvm_phase phase);
            super.connect_phase(phase);
            cov_export.connect(cov_fifo.analysis_export);
        endfunction


        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            forever begin
                cov_fifo.get(seq_item_cov);
                cg_rst.sample();

                if (!seq_item_cov.rst_n) begin
                    // Initialize pre-edge state on active reset
                    full_gate        = 0;
                    empty_gate       = 1;
                    almostfull_gate  = 0;
```

```
                    almostempty_gate = 0;
                end
                else begin
                    // 1. Sample cg_FIFO with latched pre-edge state
                    cg_FIFO.sample();

                    // 2. Latch current state for the next transaction cycle
                    full_gate        = seq_item_cov.full;
                    empty_gate       = seq_item_cov.empty;
                    almostfull_gate  = seq_item_cov.almostfull;
                    almostempty_gate = seq_item_cov.almostempty;
                end
            end
        endtask
    endclass
endpackage
```

➢ Assertions:

```
import shared_pkg::*;
module FIFO_sva(FIFO_if f_if);

`ifdef SIM

    //========================================================================
    // Asynchronous (reset) and combinational checks
    // These signals do not wait for a clock edge, so they are checked in an
    // always_comb block with deferred immediate assertions (assert final).
    // assert final evaluates the expression at the final settled value of the
    // time step, after the async reset and the continuous assigns have updated.
    //========================================================================
    always_comb begin : async_and_comb_checks

        //---------------- asynchronous reset assertions ----------------
        if (!f_if.rst_n) begin
            assert_reset_flags: assert final (f_if.empty == 1'b1 && f_if.full == 1'b0 && f_if.almostfull
== 1'b0 && f_if.almostempty == 1'b0)
                else $error("Assertion reset_flags failed!");
            cover_reset_flags: cover final (f_if.empty == 1'b1 && f_if.full == 1'b0 && f_if.almostfull ==
1'b0 && f_if.almostempty == 1'b0);

            assert_RST_wr_ptr: assert final (f_if.wr_ptr == 0)
                else $error("Assertion RST_wr_ptr failed!");
            cover_RST_wr_ptr: cover final (f_if.wr_ptr == 0);

            assert_RST_rd_ptr: assert final (f_if.rd_ptr == 0)
                else $error("Assertion RST_rd_ptr failed!");
            cover_RST_rd_ptr: cover final (f_if.rd_ptr == 0);

            assert_RST_count: assert final (f_if.count == 0)
                else $error("Assertion RST_count failed!");
            cover_RST_count: cover final (f_if.count == 0);

            assert_async_reset_seq: assert final (f_if.overflow == 1'b0 && f_if.underflow == 1'b0 &&
f_if.wr_ack == 1'b0)
                else $error("Async reset failed to clear sequential outputs!");
            cover_async_reset_seq: cover final (f_if.overflow == 1'b0 && f_if.underflow == 1'b0 &&
f_if.wr_ack == 1'b0);
        end

        //---------------- combinational flag assertions ----------------
        if (f_if.count == 0) begin
            assert_empty_flag: assert final (f_if.empty)
                else $error("Assertion empty_flag failed!");
            cover_empty_flag: cover final (f_if.empty);
```

```
        end

        if (f_if.count == f_if.FIFO_DEPTH) begin
            assert_full_flag: assert final (f_if.full)
                else $error("Assertion full_flag failed!");
            cover_full_flag: cover final (f_if.full);
        end

        if (f_if.count == (f_if.FIFO_DEPTH-1)) begin
            assert_almost_full: assert final (f_if.almostfull)
                else $error("Assertion almost_full failed!");
            cover_almost_full: cover final (f_if.almostfull);
        end

        if (f_if.count == 1) begin
            assert_almost_empty: assert final (f_if.almostempty)
                else $error("Assertion almost_empty failed!");
            cover_almost_empty: cover final (f_if.almostempty);
        end
    end

    //========================================================================
    // Sequential assertions (clocked concurrent assertions)
    //========================================================================
    property write_ack;
        @(posedge f_if.clk) disable iff (!f_if.rst_n) (f_if.wr_en) && (!f_if.full)  |=> (f_if.wr_ack);
    endproperty

    property overflow_detection;
        @(posedge f_if.clk) disable iff (!f_if.rst_n) (f_if.wr_en) && (f_if.full)  |=> (f_if.overflow);
    endproperty

    property underflow_detection;
        @(posedge f_if.clk) disable iff (!f_if.rst_n) (f_if.rd_en) && (f_if.empty)  |=> (f_if.underflow);
    endproperty

    //pointers assertions
    property wr_wraparound;
        @(posedge f_if.clk) disable iff (!f_if.rst_n) (f_if.wr_en && f_if.count < f_if.FIFO_DEPTH &&
f_if.wr_ptr == f_if.FIFO_DEPTH-1) |=> (f_if.wr_ptr == 0) ;
    endproperty

    property rd_wraparound;
        @(posedge f_if.clk) disable iff (!f_if.rst_n) (f_if.rd_en && f_if.count != 0 && f_if.rd_ptr ==
f_if.FIFO_DEPTH-1) |=> (f_if.rd_ptr==0);
    endproperty

    property threshold;
```

```
        @(posedge f_if.clk) disable iff (!f_if.rst_n) (f_if.wr_ptr < f_if.FIFO_DEPTH) && (f_if.rd_ptr <
f_if.FIFO_DEPTH) && (f_if.count <= f_if.FIFO_DEPTH);
    endproperty

    //========================================================================
    // Assertions & coverage for the sequential checks
    //========================================================================
    assert_write_ack: assert property (write_ack)
        else $error("Assertion write_ack failed!");

    cover property(write_ack);

    assert_overflow_detection: assert property(overflow_detection)
        else $error("Assertion overflow_detection failed!");

    cover property(overflow_detection);

    assert_underflow_detection: assert property(underflow_detection)
        else $error("Assertion underflow_detection failed!");

    cover property(underflow_detection);

    assert_wr_wraparound: assert property(wr_wraparound)
        else $error("Assertion wr_wraparound failed!");

    cover property(wr_wraparound);

    assert_rd_wraparound: assert property(rd_wraparound)
        else $error("Assertion rd_wraparound failed!");

    cover property(rd_wraparound);

    assert_threshold: assert property(threshold)
        else $error("Assertion threshold failed!");

    cover property(threshold);

`endif
endmodule
```

Verification_plan:

Each requirement lists how it is stimulated, what is covered and how it is checked. **Immediate assertion (assert final)** is a deferred immediate assertion in an always_comb block, used for reset and the combinational flags. **Concurrent assertion** is a clocked assert property, used for sequential and pointer behavior.

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
FIFO_1	Reset (rst_n = 0) clears wr_ptr, rd_ptr and count to 0.	Directed: FIFO_rst_seq at start (rst_n = 0, other inputs 0). Random: rst_n dist {1:=98, 0:=2}.	Pin rst_n (cg_rst): asserted, deasserted, assert edge, deassert edge.	Immediate assertion (assert final): RST_wr_ptr, RST_rd_ptr, RST_count. Golden model.
FIFO_2	Reset (rst_n = 0) clears wr_ack, overflow and underflow to 0.	Directed: FIFO_rst_seq at start (rst_n = 0, other inputs 0). Random: rst_n dist {1:=98, 0:=2}.	Pin rst_n (cg_rst).	Immediate assertion (assert final): async_reset_seq. Golden model.
FIFO_3	Reset (rst_n = 0) sets empty = 1 and full, almostfull, almostempty = 0.	Directed: FIFO_rst_seq at start (rst_n = 0, other inputs 0). Random: rst_n dist {1:=98, 0:=2}.	Pin rst_n (cg_rst).	Immediate assertion (assert final): reset_flags. Golden model.
FIFO_4	Write when not full: data stored, wr_ptr and count increment, wr_ack = 1.	Random: wr_en dist {1:=70, 0:=30}, rd_en dist {1:=30, 0:=70}. Directed: FIFO_wr_seq (wr_en = 1, rd_en = 0).	Pins wr_en, rd_en, wr_ack. Cross: cross_wr_ack.	Concurrent assertion: write_ack. Golden model.
FIFO_5	Read when not empty: data_out = memory word, rd_ptr increments, count decrements.	Random: rd_en dist {1:=30, 0:=70}, wr_en dist {1:=70, 0:=30}. Directed: FIFO_rd_seq (rd_en = 1, wr_en = 0).	Pins rd_en, wr_en, empty, almostempty. Crosses: cross_empty, cross_almost_empty.	Golden model (data_out compare).
FIFO_6	Write when full: write rejected, overflow = 1.	Directed: FIFO_wr_seq keeps writing after full. Random: wr_en dist {1:=70, 0:=30}.	Pins wr_en, rd_en, full, overflow. Crosses: cross_overflow, cross_full.	Concurrent assertion: overflow_detection. Golden model.
FIFO_7	Read when empty: read rejected, underflow = 1.	Directed: FIFO_rd_seq keeps reading after empty. Random: rd_en dist {1:=30, 0:=70}.	Pins wr_en, rd_en, empty, underflow. Crosses: cross_underflow, cross_empty.	Concurrent assertion: underflow_detection. Golden model.
FIFO_8	empty = 1 when count = 0, otherwise 0.	Random: wr_en and rd_en (70/30) move count to and from 0. Directed: FIFO_rd_seq drains the FIFO.	Pins wr_en, rd_en, empty. Cross: cross_empty.	Immediate assertion (assert final): empty_flag. Golden model.
FIFO_9	full = 1 when count = FIFO_DEPTH (8), otherwise 0.	Random: wr_en and rd_en (70/30) move count to and from FIFO_DEPTH. Directed: FIFO_wr_seq fills the FIFO.	Pins wr_en, rd_en, full. Cross: cross_full.	Immediate assertion (assert final): full_flag. Golden model.

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
FIFO_10	almostfull = 1 when count = FIFO_DEPTH - 1 (7), otherwise 0.	Random: wr_en and rd_en (70/30) reach count = 7. Directed: FIFO_wr_seq.	Pins wr_en, rd_en, almostfull. Cross: cross_almost_full.	Immediate assertion (assert final): almost_full. Golden model.
FIFO_11	almostempty = 1 when count = 1, otherwise 0.	Random: wr_en and rd_en (70/30) reach count = 1. Directed: FIFO_rd_seq.	Pins wr_en, rd_en, almostempty. Cross: cross_almost_empty.	Immediate assertion (assert final): almost_empty. Golden model.
FIFO_12	Write and read together (not full, not empty): both succeed, count unchanged.	Directed: FIFO_wr_rd_seq2 (wr_en = 1, rd_en = 1). Random: wr_en and rd_en constraints (70/30).	Crosses with wr_en = 1, rd_en = 1: cross_full, cross_almost_full, cross_empty, cross_almost_empty.	Concurrent assertion: write_ack. Golden model.
FIFO_13	Write and read together when empty: only the write succeeds, count + 1, underflow = 1.	Directed: FIFO_wr_rd_seq2 while the FIFO is empty (after reset or drain).	Bins wr_en = 1, rd_en = 1 in cross_empty and cross_underflow.	Concurrent assertions: underflow_detection, write_ack. Golden model.
FIFO_14	Write and read together when full: only the read succeeds, count - 1, overflow = 1.	Directed: FIFO_wr_rd_seq2 while the FIFO is full (after FIFO_wr_seq).	Bins wr_en = 1, rd_en = 1 in cross_full and cross_overflow.	Concurrent assertion: overflow_detection. Golden model.
FIFO_15	wr_ptr wraps from FIFO_DEPTH - 1 (7) to 0 on an accepted write.	Directed: FIFO_wr_seq and the 10-write bursts of FIFO_wr_rd_seq1.	None (cover property wr_wraparound).	Concurrent assertion: wr_wraparound. Golden model.
FIFO_16	rd_ptr wraps from FIFO_DEPTH - 1 (7) to 0 on an accepted read.	Directed: FIFO_rd_seq and the 10-read bursts of FIFO_wr_rd_seq1.	None (cover property rd_wraparound).	Concurrent assertion: rd_wraparound. Golden model.
FIFO_17	wr_ptr and rd_ptr stay <= FIFO_DEPTH - 1 (7) and count stays <= FIFO_DEPTH (8).	All sequences, 2301 transactions.	None (cover property threshold).	Concurrent assertion: threshold.
FIFO_18	Inputs are applied and outputs sampled at negedge clk, so each transaction pairs inputs with their outputs.	Directed: driver drives at negedge clk; monitor samples at negedge clk.	-	Golden model comparison on every transaction.
FIFO_19	End of test: correct and failed counts are reported.	Directed: FIFO_test runs rst_seq, wr_seq, rd_seq, wr_rd_seq2, wr_rd_seq1, then drops the objection.	-	Scoreboard report_phase (correct_count, error_count).
FIFO_20	Fill: continuous writes take the FIFO through almostfull and full; extra writes give overflow.	Directed: FIFO_wr_seq (wr_en = 1, rd_en = 0), 100 items.	cross_almost_full, cross_full, cross_overflow, cross_wr_ack.	Concurrent assertions: write_ack, overflow_detection. Immediate assertions (assert final): almost_full, full_flag. Golden model.

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
FIFO_21	Drain: continuous reads take the FIFO through almostempty and empty; extra reads give underflow.	Directed: FIFO_rd_seq (rd_en = 1, wr_en = 0), 100 items.	cross_almost_empty, cross_empty, cross_underflow.	Concurrent assertion: underflow_detection. Immediate assertions (assert final): almost_empty, empty_flag. Golden model.
FIFO_22	Continuous write and read together through the normal, empty and full cases.	Directed: FIFO_wr_rd_seq2, 100 items (wr_en == 1 && rd_en == 1).	Crosses with wr_en = 1, rd_en = 1: full, almostfull, empty, almostempty, overflow, underflow, wr_ack.	Concurrent assertions: write_ack, overflow_detection, underflow_detection. Golden model.
FIFO_23	Alternating bursts: 10 writes then 10 reads, repeated; count sweeps 0 to FIFO_DEPTH and both pointers wrap.	Directed: FIFO_wr_rd_seq1, 100 x (10 writes, 10 reads).	Flag crosses; cover properties wr_wraparound, rd_wraparound.	Concurrent assertions: wr_wraparound, rd_wraparound, threshold. Golden model.
FIFO_24	Reset during operation (full, partly filled or empty): DUT returns to the reset state and keeps working.	Random: rst_n dist {1:=98, 0:=2} in every sequence.	Pin rst_n (cg_rst): assert edge, deassert edge.	Immediate assertions (assert final): reset checks. Golden model.
FIFO_25	Data read equals data written, in order, including after pointer wraps.	Random: data_in 16-bit random on every item.	- (data_in and data_out are not covered).	Golden model (data_out compare).
FIFO_26	Idle (wr_en = 0, rd_en = 0): count, pointers and data_out hold; wr_ack, overflow, underflow = 0.	Random: wr_en and rd_en constraints (70/30); idle cycles occur by chance.	Crosses with wr_en = 0, rd_en = 0: cross_full, cross_almost_full, cross_empty, cross_almost_empty.	Golden model.
FIFO_27	A rejected write (full) or read (empty) leaves the state unchanged.	Directed: FIFO_wr_seq after full, FIFO_rd_seq after empty.	cross_overflow, cross_underflow.	Concurrent assertions: overflow_detection, underflow_detection. Golden model.

Src_files_list:

src_files.list

```
shared_pkg.sv
FIFO_if.sv
FIFO_config.sv
FIFO_seq_item.sv
FIFO_driver.sv
FIFO_sequencer.sv
FIFO_monitor.sv
FIFO_coverage.sv
FIFO_scoreboard.sv
FIFO_agent.sv
FIFO_env.sv
FIFO_rst_seq.sv
FIFO_wr_seq.sv
FIFO_rd_seq.sv
FIFO_wr_rd_seq.sv
FIFO_test.sv
FIFO_top.sv
```

Src_dut_list:

src_dut.list

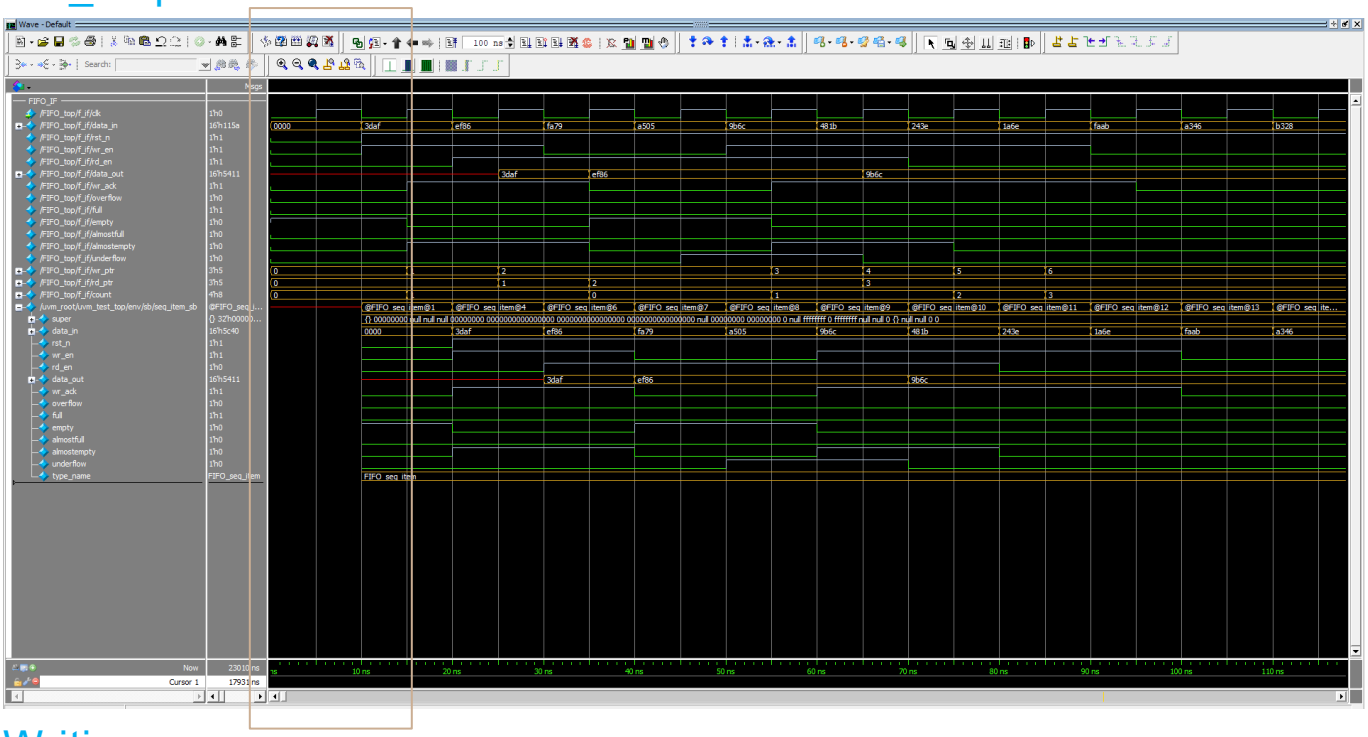
```
FIFO.sv
FIFO_sva.sv
FIFO_top.sv
```

Do_file:

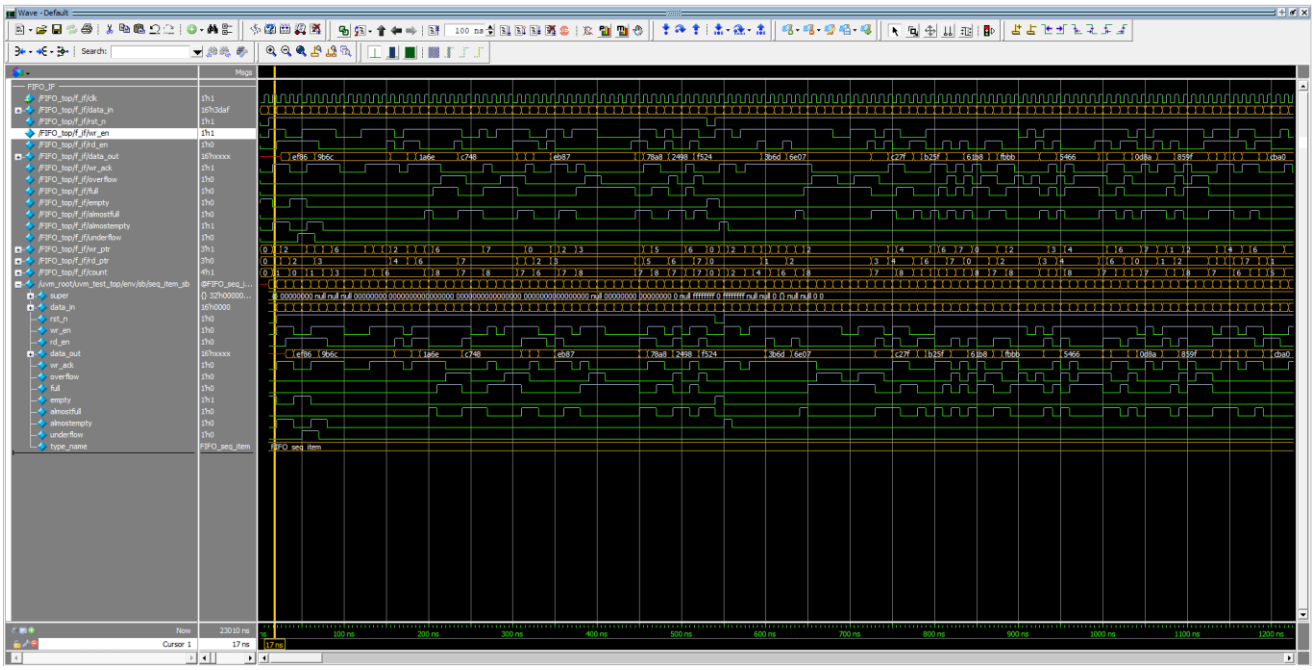
```
vlib work
vlog -f src_files.list
vlog -f src_dut.list +cover=bcesft +define+SIM
vsim -voptargs=+acc -cover +define+SIM work.FIFO_top -classdebug -uvmcontrol=all
-assertdebug
add wave -divider FIFO_IF
add wave /FIFO_top/f_if/*
run 0
add wave -position insertpoint sim:/uvm_root/uvm_test_top/env/sb/seq_item_sb
coverage save FIFO_top.ucdb -onexit
run -all
##vcover report FIFO_top.ucdb -details -annotate -all -output coverage.txt
```

Waveform:

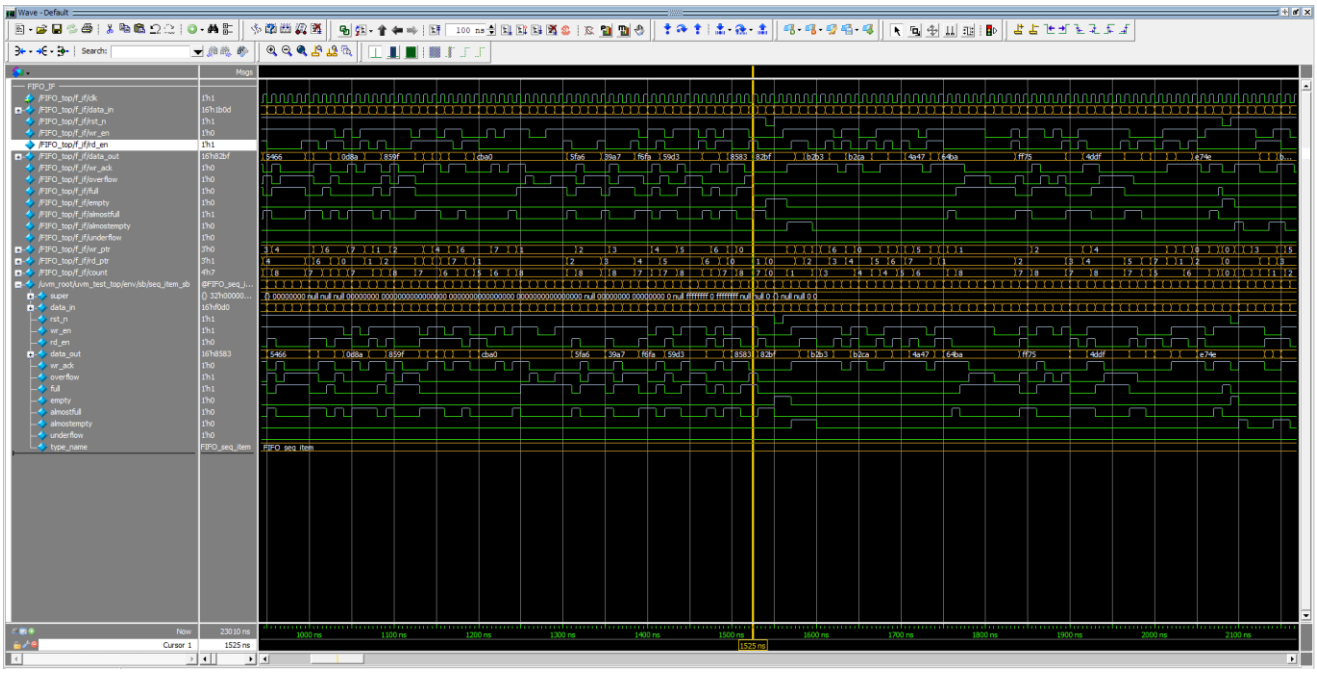
Rst_seq:



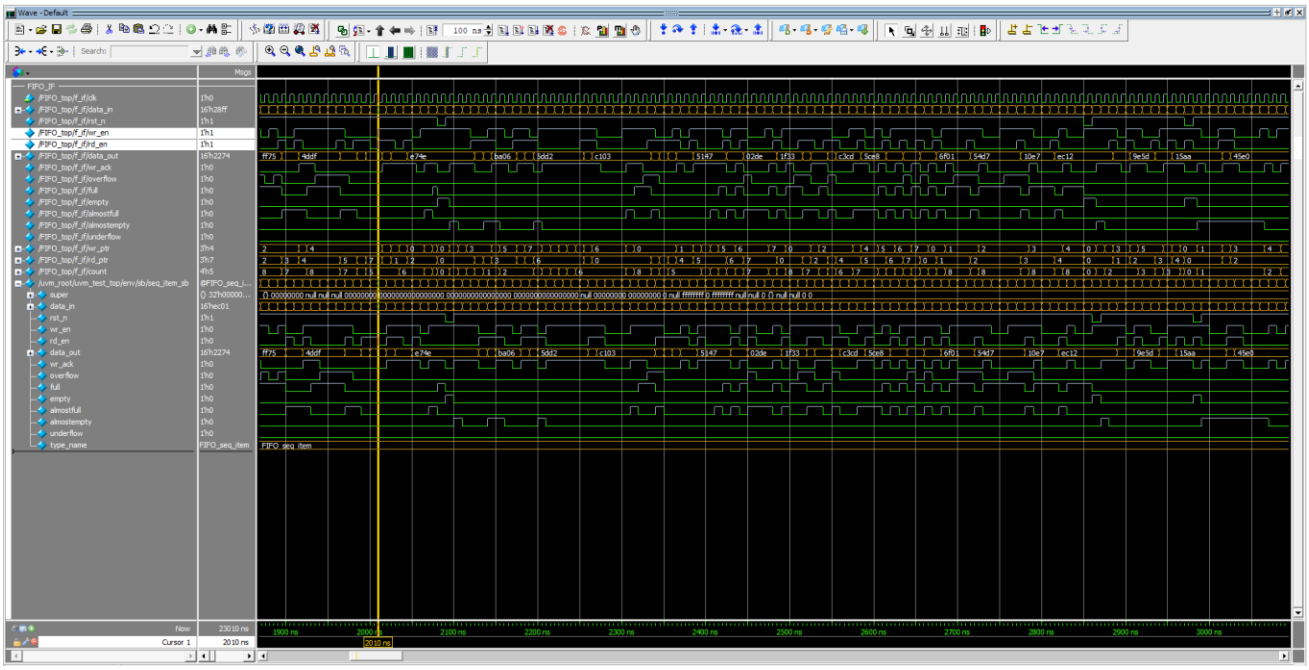
Writing:

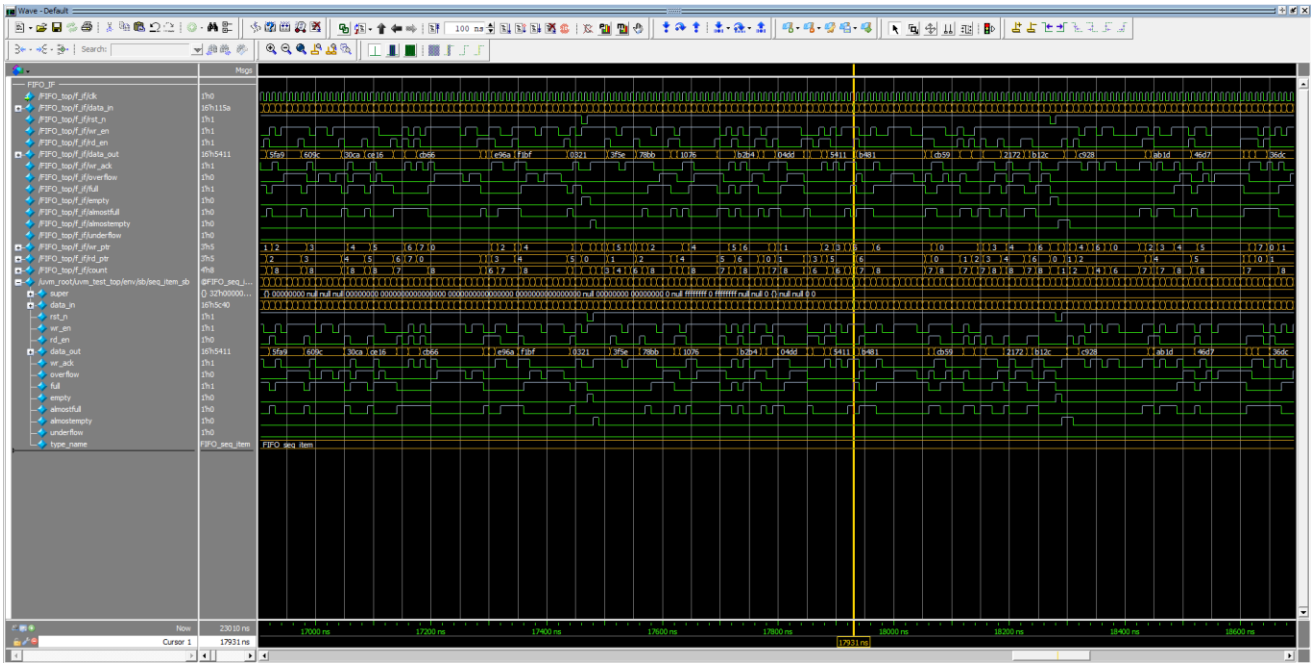


Reading:



Writing & reading:

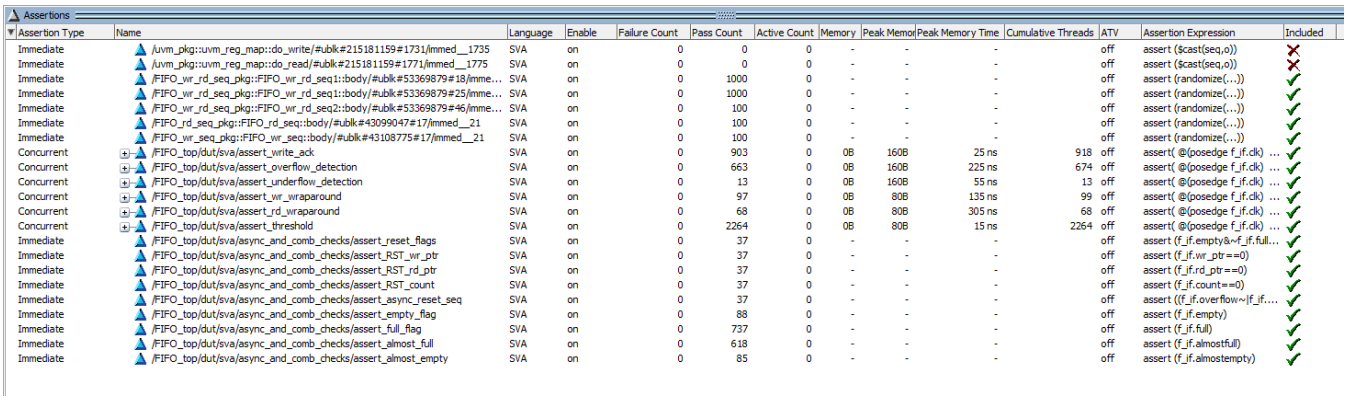




Transcript:

```
# ----------------------------------------------------------------
#
#   ***********       IMPORTANT RELEASE NOTES         ************
#
#   You are using a version of the UVM library that has been compiled
#   with `UVM_NO_DEPRECATED undefined.
#   See http://www.eda.org/svdb/view.php?id=3313 for more details.
#
#   You are using a version of the UVM library that has been compiled
#   with `UVM_OBJECT_MUST_HAVE_CONSTRUCTOR undefined.
#   See http://www.eda.org/svdb/view.php?id=3770 for more details.
#
#       (Specify +UVM_NO_RELNOTES to turn off this notice)
#
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(277) @ 0: reporter [Questa UVM] QUESTA_UVM-1.2.3
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(278) @ 0: reporter [Questa UVM]  questa_uvm::init(all)
# UVM_INFO @ 0: reporter [RNTST] Running test FIFO_test...
# UVM_INFO FIFO_test.sv(48) @ 0: uvm_test_top [run_phase] Reset Asserted
# ********************************************************************************
# * Questa UVM Transaction Recording Turned ON.                                  *
# * recording_detail has been set.                                               *
# *  To turn off, set 'recording_detail' to off:                                 *
# * uvm_config_db#(int)            ::set(null, "", "recording_detail", 0); *
# * uvm_config_db#(uvm_bitstream_t)::set(null, "", "recording_detail", 0); *
# ********************************************************************************
# UVM_INFO FIFO_test.sv(50) @ 10: uvm_test_top [run_phase] Reset Deasserted
# UVM_INFO FIFO_test.sv(52) @ 10: uvm_test_top [run_phase] Stimulus Generation Started
# UVM_INFO FIFO_test.sv(54) @ 10: uvm_test_top [run_phase] Writing Started
# UVM_INFO FIFO_test.sv(57) @ 1010: uvm_test_top [run_phase] Reading Started
# UVM_INFO FIFO_test.sv(60) @ 2010: uvm_test_top [run_phase] Writing and Reading simultaneously Started
# UVM_INFO FIFO_test.sv(63) @ 3010: uvm_test_top [run_phase] Writing and Reading Started
# UVM_INFO FIFO_test.sv(66) @ 23010: uvm_test_top [run_phase] Stimulus Generation Ended
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 23010: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO FIFO_scoreboard.sv(142) @ 23010: uvm_test_top.env.sb [report_phase] Total successful transactions:2301
# UVM_INFO FIFO_scoreboard.sv(143) @ 23010: uvm_test_top.env.sb [report_phase] Total failed transactions:0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :   14
# UVM_WARNING :    0
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [Questa UVM]     2
# [RNTST]     1
# [TEST_DONE]     1
# [report_phase]     2
# [run_phase]     8
# ** Note: $finish    : D:/questa/win64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
#    Time: 23010 ns  Iteration: 61  Instance: /FIFO_top
# 1
# Break in Task uvm_pkg/uvm_root::run_test at D:/questa/win64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430
```

- **Assertions:**



Assertions table:

Feature	Assertion
Flags at reset	`if (!rst_n) assert final (empty == 1 && full == 0 && almostfull == 0 && almostempty == 0)`
`wr_ptr` at reset	`if (!rst_n) assert final (wr_ptr == 0)`
`rd_ptr` at reset	`if (!rst_n) assert final (rd_ptr == 0)`
`count` at reset	`if (!rst_n) assert final (count == 0)`
Sequential outputs at reset	`if (!rst_n) assert final (overflow == 0 && underflow == 0 && wr_ack == 0)`
`empty` flag	`if (count == 0) assert final (empty)`
`full` flag	`if (count == FIFO_DEPTH) assert final (full)`
`almostfull` flag	`if (count == FIFO_DEPTH-1) assert final (almostfull)`
`almostempty` flag	`if (count == 1) assert final (almostempty)`
Write acknowledge	`@(posedge clk) disable iff (!rst_n) (wr_en && !full) |=> wr_ack`
Overflow	`@(posedge clk) disable iff (!rst_n) (wr_en && full) |=> overflow`
Underflow	`@(posedge clk) disable iff (!rst_n) (rd_en && empty) |=> underflow`
Write pointer wraparound	`@(posedge clk) disable iff (!rst_n) (wr_en && count < FIFO_DEPTH && wr_ptr == FIFO_DEPTH-1) |=> wr_ptr == 0`
Read pointer wraparound	`@(posedge clk) disable iff (!rst_n) (rd_en && count != 0 && rd_ptr == FIFO_DEPTH-1) |=> rd_ptr == 0`
Pointer and counter bounds	`@(posedge clk) disable iff (!rst_n) (wr_ptr < FIFO_DEPTH) && (rd_ptr < FIFO_DEPTH) && (count <= FIFO_DEPTH)`

- **Code_directives:**

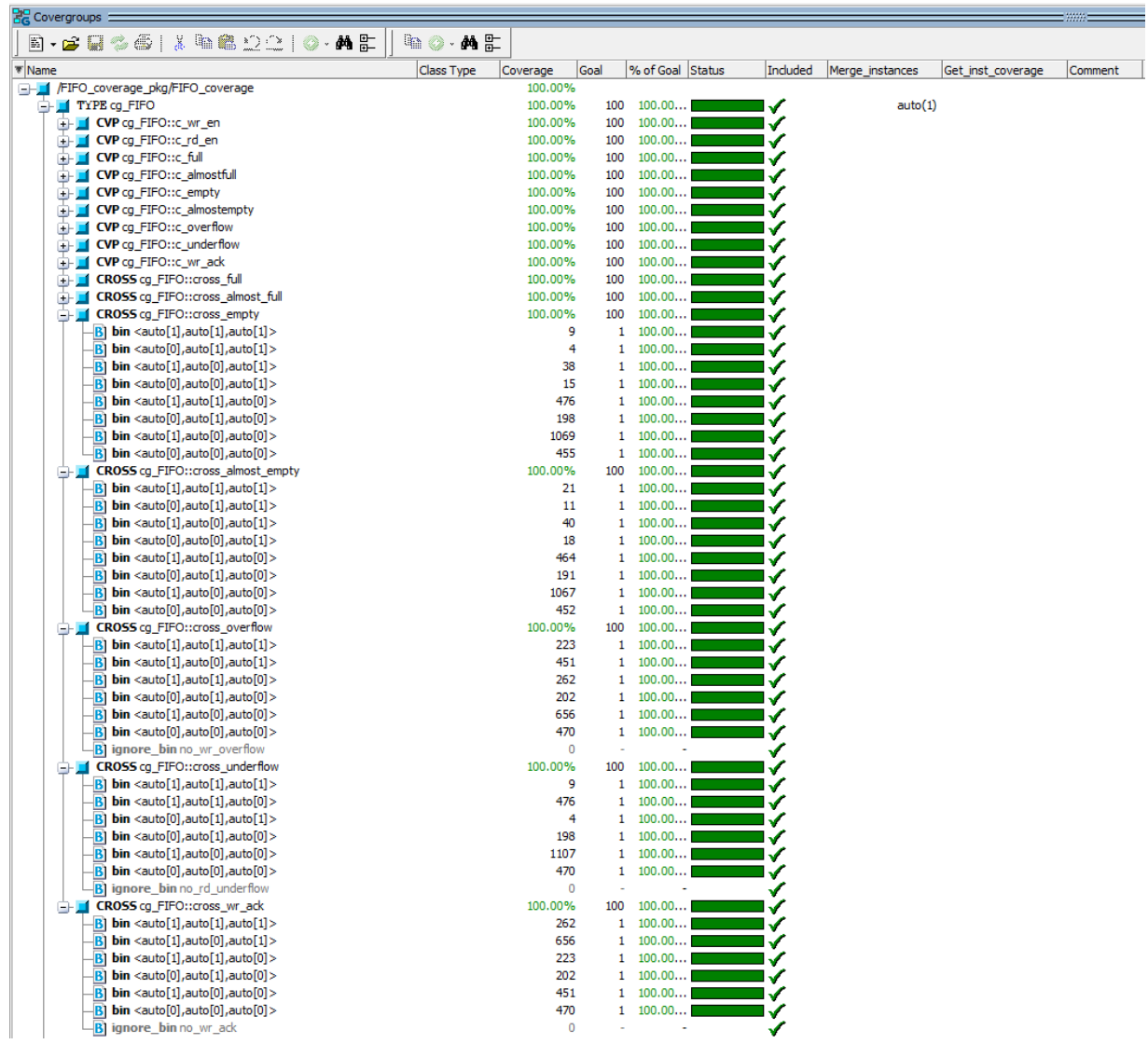
Cover Directives

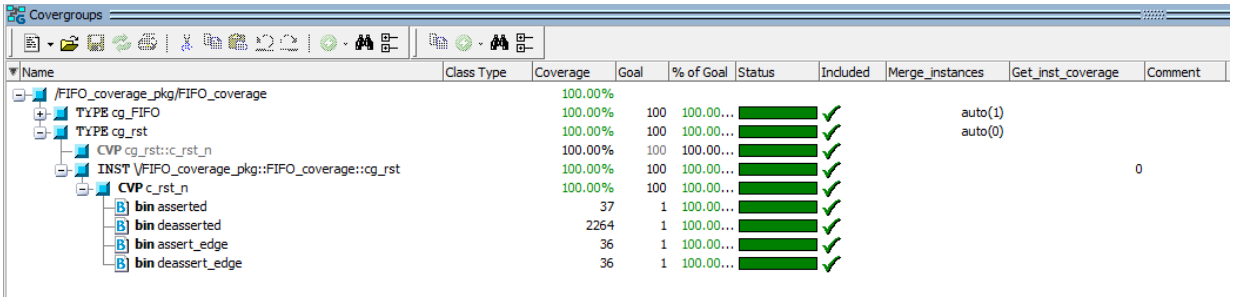
Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory	Peak Memory	Peak Memory Time	Cumulative Threads
/FIFO_top/dut/sva/cover__threshold	SVA	✓	Off	2264	1	Unlimited	1	100%		✓	0	80	15 ns	2264
/FIFO_top/dut/sva/cover__rd_wraparound	SVA	✓	Off	68	1	Unlimited	1	100%		✓	0	80	305 ns	68
/FIFO_top/dut/sva/cover__wr_wraparound	SVA	✓	Off	97	1	Unlimited	1	100%		✓	0	80	135 ns	99
/FIFO_top/dut/sva/cover__underflow_detection	SVA	✓	Off	13	1	Unlimited	1	100%		✓	0	160	55 ns	13
/FIFO_top/dut/sva/cover__overflow_detection	SVA	✓	Off	663	1	Unlimited	1	100%		✓	0	160	225 ns	674
/FIFO_top/dut/sva/cover__write_ack	SVA	✓	Off	903	1	Unlimited	1	100%		✓	0	160	25 ns	918
/FIFO_top/dut/sva/async_and_comb_checks/cov...	SVA	✓	Off	37	1	Unlimited	1	100%		✓	0	0	0 ns	0
/FIFO_top/dut/sva/async_and_comb_checks/cov...	SVA	✓	Off	37	1	Unlimited	1	100%		✓	0	0	0 ns	0
/FIFO_top/dut/sva/async_and_comb_checks/cov...	SVA	✓	Off	37	1	Unlimited	1	100%		✓	0	0	0 ns	0
/FIFO_top/dut/sva/async_and_comb_checks/cov...	SVA	✓	Off	37	1	Unlimited	1	100%		✓	0	0	0 ns	0
/FIFO_top/dut/sva/async_and_comb_checks/cov...	SVA	✓	Off	37	1	Unlimited	1	100%		✓	0	0	0 ns	0
/FIFO_top/dut/sva/async_and_comb_checks/cov...	SVA	✓	Off	88	1	Unlimited	1	100%		✓	0	0	0 ns	0
/FIFO_top/dut/sva/async_and_comb_checks/cov...	SVA	✓	Off	737	1	Unlimited	1	100%		✓	0	0	0 ns	0
/FIFO_top/dut/sva/async_and_comb_checks/cov...	SVA	✓	Off	618	1	Unlimited	1	100%		✓	0	0	0 ns	0
/FIFO_top/dut/sva/async_and_comb_checks/cov...	SVA	✓	Off	85	1	Unlimited	1	100%		✓	0	0	0 ns	0

- **Coverage_groups:**

Covergroups

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment
/FIFO_coverage_pkg/FIFO_coverage		100.00%							
TYPE cg_FIFO		100.00%	100	100.00...		✓		auto(1)	
CVP cg_FIFO::c_wr_en		100.00%	100	100.00...		✓			
bin auto[0]		672	1	100.00...		✓			
bin auto[1]		1592	1	100.00...		✓			
CVP cg_FIFO::c_rd_en		100.00%	100	100.00...		✓			
bin auto[0]		1577	1	100.00...		✓			
bin auto[1]		687	1	100.00...		✓			
CVP cg_FIFO::c_full		100.00%	100	100.00...		✓			
bin auto[0]		1294	1	100.00...		✓			
bin auto[1]		970	1	100.00...		✓			
CVP cg_FIFO::c_almostfull		100.00%	100	100.00...		✓			
bin auto[0]		1621	1	100.00...		✓			
bin auto[1]		643	1	100.00...		✓			
CVP cg_FIFO::c_empty		100.00%	100	100.00...		✓			
bin auto[0]		2198	1	100.00...		✓			
bin auto[1]		66	1	100.00...		✓			
CVP cg_FIFO::c_almostempty		100.00%	100	100.00...		✓			
bin auto[0]		2174	1	100.00...		✓			
bin auto[1]		90	1	100.00...		✓			
CVP cg_FIFO::c_overflow		100.00%	100	100.00...		✓			
bin auto[0]		1590	1	100.00...		✓			
bin auto[1]		674	1	100.00...		✓			
CVP cg_FIFO::c_underflow		100.00%	100	100.00...		✓			
bin auto[0]		2251	1	100.00...		✓			
bin auto[1]		13	1	100.00...		✓			
CVP cg_FIFO::c_wr_ack		100.00%	100	100.00...		✓			
bin auto[0]		1346	1	100.00...		✓			
bin auto[1]		918	1	100.00...		✓			
CROSS cg_FIFO::cross_full		100.00%	100	100.00...		✓			
bin <auto[1],auto[1],auto[1]>		223	1	100.00...		✓			
bin <auto[0],auto[1],auto[1]>		91	1	100.00...		✓			
bin <auto[1],auto[0],auto[1]>		451	1	100.00...		✓			
bin <auto[0],auto[0],auto[1]>		205	1	100.00...		✓			
bin <auto[1],auto[1],auto[0]>		262	1	100.00...		✓			
bin <auto[0],auto[1],auto[0]>		111	1	100.00...		✓			
bin <auto[1],auto[0],auto[0]>		656	1	100.00...		✓			
bin <auto[0],auto[0],auto[0]>		265	1	100.00...		✓			
CROSS cg_FIFO::cross_almost_full		100.00%	100	100.00...		✓			
bin <auto[1],auto[1],auto[1]>		125	1	100.00...		✓			
bin <auto[0],auto[1],auto[1]>		59	1	100.00...		✓			
bin <auto[1],auto[0],auto[1]>		330	1	100.00...		✓			
bin <auto[0],auto[0],auto[1]>		129	1	100.00...		✓			
bin <auto[1],auto[1],auto[0]>		360	1	100.00...		✓			
bin <auto[0],auto[1],auto[0]>		143	1	100.00...		✓			
bin <auto[1],auto[0],auto[0]>		777	1	100.00...		✓			
bin <auto[0],auto[0],auto[0]>		341	1	100.00...		✓			
CROSS cg_FIFO::cross_empty		100.00%	100	100.00...		✓			
CROSS cg_FIFO::cross_almost_empty		100.00%	100	100.00...		✓			
CROSS cg_FIFO::cross_overflow		100.00%	100	100.00...		✓			
CROSS cg_FIFO::cross_underflow		100.00%	100	100.00...		✓			
CROSS cg_FIFO::cross_wr_ack		100.00%	100	100.00...		✓			


Covergroups
Name
Class Type
Coverage
Goal
% of Goal
Status
Included
Merge_instances
Get_inst_coverage
Comment
/FIFO_coverage_pkg/FIFO_coverage 100.00%
TYPE cg_FIFO 100.00% 100 100.00... auto(1)
CVP cg_FIFO::c_wr_en 100.00% 100 100.00...
CVP cg_FIFO::c_rd_en 100.00% 100 100.00...
CVP cg_FIFO::c_full 100.00% 100 100.00...
CVP cg_FIFO::c_almostfull 100.00% 100 100.00...
CVP cg_FIFO::c_empty 100.00% 100 100.00...
CVP cg_FIFO::c_almostempty 100.00% 100 100.00...
CVP cg_FIFO::c_overflow 100.00% 100 100.00...
CVP cg_FIFO::c_underflow 100.00% 100 100.00...
CVP cg_FIFO::c_wr_ack 100.00% 100 100.00...
CROSS cg_FIFO::cross_full 100.00% 100 100.00...
CROSS cg_FIFO::cross_almost_full 100.00% 100 100.00...
CROSS cg_FIFO::cross_empty 100.00% 100 100.00...
bin <auto[1],auto[1],auto[1]> 9 1 100.00...
bin <auto[0],auto[1],auto[1]> 4 1 100.00...
bin <auto[1],auto[0],auto[1]> 38 1 100.00...
bin <auto[0],auto[0],auto[1]> 15 1 100.00...
bin <auto[1],auto[1],auto[0]> 476 1 100.00...
bin <auto[0],auto[1],auto[0]> 198 1 100.00...
bin <auto[1],auto[0],auto[0]> 1069 1 100.00...
bin <auto[0],auto[0],auto[0]> 455 1 100.00...
CROSS cg_FIFO::cross_almost_empty 100.00% 100 100.00...
bin <auto[1],auto[1],auto[1]> 21 1 100.00...
bin <auto[0],auto[1],auto[1]> 11 1 100.00...
bin <auto[1],auto[0],auto[1]> 40 1 100.00...
bin <auto[0],auto[0],auto[1]> 18 1 100.00...
bin <auto[1],auto[1],auto[0]> 464 1 100.00...
bin <auto[0],auto[1],auto[0]> 191 1 100.00...
bin <auto[1],auto[0],auto[0]> 1067 1 100.00...
bin <auto[0],auto[0],auto[0]> 452 1 100.00...
CROSS cg_FIFO::cross_overflow 100.00% 100 100.00...
bin <auto[1],auto[1],auto[1]> 223 1 100.00...
bin <auto[1],auto[0],auto[1]> 451 1 100.00...
bin <auto[1],auto[1],auto[0]> 262 1 100.00...
bin <auto[0],auto[1],auto[0]> 202 1 100.00...
bin <auto[1],auto[0],auto[0]> 656 1 100.00...
bin <auto[0],auto[0],auto[0]> 470 1 100.00...
ignore_bin no_wr_overflow 0 - -
CROSS cg_FIFO::cross_underflow 100.00% 100 100.00...
bin <auto[1],auto[1],auto[1]> 9 1 100.00...
bin <auto[1],auto[1],auto[0]> 476 1 100.00...
bin <auto[0],auto[1],auto[1]> 4 1 100.00...
bin <auto[0],auto[1],auto[0]> 198 1 100.00...
bin <auto[1],auto[0],auto[0]> 1107 1 100.00...
bin <auto[0],auto[0],auto[0]> 470 1 100.00...
ignore_bin no_rd_underflow 0 - -
CROSS cg_FIFO::cross_wr_ack 100.00% 100 100.00...
bin <auto[1],auto[1],auto[1]> 262 1 100.00...
bin <auto[1],auto[0],auto[1]> 656 1 100.00...
bin <auto[1],auto[1],auto[0]> 223 1 100.00...
bin <auto[0],auto[1],auto[0]> 202 1 100.00...
bin <auto[1],auto[0],auto[0]> 451 1 100.00...
bin <auto[0],auto[0],auto[0]> 470 1 100.00...
ignore_bin no_wr_ack 0 - -


Covergroups
Name
Class Type
Coverage
Goal
% of Goal
Status
Included
Merge_instances
Get_inst_coverage
Comment
/FIFO_coverage_pkg/FIFO_coverage 100.00%
TYPE cg_FIFO 100.00% 100 100.00... auto(1)
TYPE cg_rst 100.00% 100 100.00... auto(0)
CVP cg_rst::c_rst_n 100.00% 100 100.00...
INST \/FIFO_coverage_pkg::FIFO_coverage::cg_rst 100.00% 100 100.00... 0
CVP c_rst_n 100.00% 100 100.00...
bin asserted 37 1 100.00...
bin deasserted 2264 1 100.00...
bin assert_edge 36 1 100.00...
bin deassert_edge 36 1 100.00...

Coverage_report:

Sequential domain coverage

```
================================================================================
=== Instance: /FIFO_top/dut/sva
=== Design Unit: work.FIFO_sva
================================================================================

Assertion Coverage:
    Assertions                      15        15         0   100.00%

--------------------------------------------------------------------------------------------------------------------
-------------------------
Name                 File(Line)                   Failure      Pass     Vacuous     Disable     Attempt
Active Peak Active ATV
                                                  Count        Count    Count       Count       Count
Count  Count
--------------------------------------------------------------------------------------------------------------------
-------------------------
/FIFO_top/dut/sva/async_and_comb_checks/assert_reset_flags
                     FIFO_sva.sv(17)                    0         37          0           0          37
0           0 off
/FIFO_top/dut/sva/async_and_comb_checks/assert_RST_wr_ptr
                     FIFO_sva.sv(21)                    0         37          0           0          37
0           0 off
/FIFO_top/dut/sva/async_and_comb_checks/assert_RST_rd_ptr
                     FIFO_sva.sv(25)                    0         37          0           0          37
0           0 off
/FIFO_top/dut/sva/async_and_comb_checks/assert_RST_count
                     FIFO_sva.sv(29)                    0         37          0           0          37
0           0 off
/FIFO_top/dut/sva/async_and_comb_checks/assert_async_reset_seq
                     FIFO_sva.sv(33)                    0         37          0           0          37
0           0 off
/FIFO_top/dut/sva/async_and_comb_checks/assert_empty_flag
                     FIFO_sva.sv(40)                    0         88          0           0          88
0           0 off
/FIFO_top/dut/sva/async_and_comb_checks/assert_full_flag
                     FIFO_sva.sv(46)                    0        737          0           0         737
0           0 off
/FIFO_top/dut/sva/async_and_comb_checks/assert_almost_full
                     FIFO_sva.sv(52)                    0        618          0           0         618
0           0 off
/FIFO_top/dut/sva/async_and_comb_checks/assert_almost_empty
                     FIFO_sva.sv(58)                    0         85          0           0          85
0           0 off
/FIFO_top/dut/sva/assert_write_ack
                     FIFO_sva.sv(95)                    0        903       1346          52        2301
0           2 off
/FIFO_top/dut/sva/assert_overflow_detection
                     FIFO_sva.sv(100)                   0        663       1590          48        2301
0           2 off
/FIFO_top/dut/sva/assert_underflow_detection
                     FIFO_sva.sv(105)                   0         13       2251          37        2301
0           2 off
```

```
/FIFO_top/dut/sva/assert_wr_wraparound
                     FIFO_sva.sv(110)                    0         97       2165          39       2301
0           1 off
/FIFO_top/dut/sva/assert_rd_wraparound
                     FIFO_sva.sv(115)                    0         68       2196          37       2301
0           1 off
/FIFO_top/dut/sva/assert_threshold
                     FIFO_sva.sv(120)                    0       2264          0          37       2301
0           1 off
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        10        10         0   100.00%

================================Branch Details================================

Branch Coverage for instance /FIFO_top/dut/sva

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File FIFO_sva.sv
------------------------------------IF Branch------------------------------------
    16                                     2028     Count coming in to IF
    16              1                        74             if (!f_if.rst_n) begin

                                           1954     All False Count
Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    39                                     2028     Count coming in to IF
    39              1                        88             if (f_if.count == 0) begin

                                           1940     All False Count
Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    45                                     2028     Count coming in to IF
    45              1                       753             if (f_if.count == f_if.FIFO_DEPTH) begin

                                           1275     All False Count
Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    51                                     2028     Count coming in to IF
    51              1                       629             if (f_if.count == (f_if.FIFO_DEPTH-1))
begin

                                           1399     All False Count
```

```
Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    57                                     2028     Count coming in to IF
    57              1                        86             if (f_if.count == 1) begin

                                           1942     All False Count
Branch totals: 2 hits of 2 branches = 100.00%


Condition Coverage:
    Enabled Coverage              Bins    Covered    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Conditions                       4         4         0   100.00%

================================Condition Details================================

Condition Coverage for instance /FIFO_top/dut/sva --

  File FIFO_sva.sv
----------------Focused Condition View-------------------
Line       39 Item    1  (f_if.count == 0)
Condition totals: 1 of 1 input term covered = 100.00%

         Input Term   Covered  Reason for no coverage   Hint
        -----------  --------  -----------------------  --------------
  (f_if.count == 0)         Y

     Rows:       Hits  FEC Target            Non-masking condition(s)
 ---------  ---------  --------------------  -------------------------
  Row   1:          1  (f_if.count == 0)_0   -
  Row   2:          1  (f_if.count == 0)_1   -

----------------Focused Condition View-------------------
Line       45 Item    1  (f_if.count == f_if.FIFO_DEPTH)
Condition totals: 1 of 1 input term covered = 100.00%

                     Input Term   Covered  Reason for no coverage   Hint
                    -----------  --------  -----------------------  --------------
  (f_if.count == f_if.FIFO_DEPTH)         Y

     Rows:       Hits  FEC Target                         Non-masking condition(s)
 ---------  ---------  --------------------               -------------------------
  Row   1:          1  (f_if.count == f_if.FIFO_DEPTH)_0  -
  Row   2:          1  (f_if.count == f_if.FIFO_DEPTH)_1  -

----------------Focused Condition View-------------------
```

```
Line       51 Item    1  (f_if.count == (f_if.FIFO_DEPTH - 1))
Condition totals: 1 of 1 input term covered = 100.00%

                            Input Term   Covered  Reason for no coverage   Hint
                           -----------  --------  ----------------------  --------------
  (f_if.count == (f_if.FIFO_DEPTH - 1))         Y

     Rows:       Hits  FEC Target                               Non-masking condition(s)
 ---------  ---------  --------------------                     -------------------------
  Row   1:          1  (f_if.count == (f_if.FIFO_DEPTH - 1))_0  -
  Row   2:          1  (f_if.count == (f_if.FIFO_DEPTH - 1))_1  -

----------------Focused Condition View-------------------
Line       57 Item    1  (f_if.count == 1)
Condition totals: 1 of 1 input term covered = 100.00%

         Input Term   Covered  Reason for no coverage   Hint
        -----------  --------  ----------------------  --------------
  (f_if.count == 1)         Y

     Rows:       Hits  FEC Target            Non-masking condition(s)
 ---------  ---------  --------------------  -------------------------
  Row   1:          1  (f_if.count == 1)_0   -
  Row   2:          1  (f_if.count == 1)_1   -



Directive Coverage:
    Directives                      15        15         0   100.00%

DIRECTIVE COVERAGE:
--------------------------------------------------------------------------------------------
Name                                    Design Design   Lang File(Line)      Hits Status
                                        Unit   UnitType
--------------------------------------------------------------------------------------------
/FIFO_top/dut/sva/cover__threshold      FIFO_sva Verilog  SVA  FIFO_sva.sv(123)2264 Covered
/FIFO_top/dut/sva/cover__rd_wraparound  FIFO_sva Verilog  SVA  FIFO_sva.sv(118)  68 Covered
/FIFO_top/dut/sva/cover__wr_wraparound  FIFO_sva Verilog  SVA  FIFO_sva.sv(113)  97 Covered
/FIFO_top/dut/sva/cover__underflow_detection
                                        FIFO_sva Verilog  SVA  FIFO_sva.sv(108)  13 Covered
/FIFO_top/dut/sva/cover__overflow_detection
                                        FIFO_sva Verilog  SVA  FIFO_sva.sv(103) 663 Covered
/FIFO_top/dut/sva/cover__write_ack      FIFO_sva Verilog  SVA  FIFO_sva.sv(98)  903 Covered
/FIFO_top/dut/sva/async_and_comb_checks/cover_reset_flags
                                        FIFO_sva Verilog  SVA  FIFO_sva.sv(19)   37 Covered
/FIFO_top/dut/sva/async_and_comb_checks/cover_RST_wr_ptr
                                        FIFO_sva Verilog  SVA  FIFO_sva.sv(23)   37 Covered
```

```
/FIFO_top/dut/sva/async_and_comb_checks/cover_RST_rd_ptr
                                     FIFO_sva Verilog  SVA  FIFO_sva.sv(27)   37 Covered
/FIFO_top/dut/sva/async_and_comb_checks/cover_RST_count
                                     FIFO_sva Verilog  SVA  FIFO_sva.sv(31)   37 Covered
/FIFO_top/dut/sva/async_and_comb_checks/cover_async_reset_seq
                                     FIFO_sva Verilog  SVA  FIFO_sva.sv(35)   37 Covered
/FIFO_top/dut/sva/async_and_comb_checks/cover_empty_flag
                                     FIFO_sva Verilog  SVA  FIFO_sva.sv(42)   88 Covered
/FIFO_top/dut/sva/async_and_comb_checks/cover_full_flag
                                     FIFO_sva Verilog  SVA  FIFO_sva.sv(48)  737 Covered
/FIFO_top/dut/sva/async_and_comb_checks/cover_almost_full
                                     FIFO_sva Verilog  SVA  FIFO_sva.sv(54)  618 Covered
/FIFO_top/dut/sva/async_and_comb_checks/cover_almost_empty
                                     FIFO_sva Verilog  SVA  FIFO_sva.sv(60)   85 Covered
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       1         1         0   100.00%

================================Statement Details================================

Statement Coverage for instance /FIFO_top/dut/sva --

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File FIFO_sva.sv
    2                                      module FIFO_sva(FIFO_if f_if);

    3

    4                                      `ifdef SIM

    5

    6
//=============================================================================

    7                                          // Asynchronous (reset) and combinational
checks

    8                                          // These signals do not wait for a clock edge,
so they are checked in an

    9                                          // always_comb block with deferred immediate
assertions (assert final).

    10                                         // assert final evaluates the expression at
the final settled value of the
```

```
    11                                                  // time step, after the async reset and the
continuous assigns have updated.

    12
//================================================================================

    13              1                       2028          always_comb begin : async_and_comb_checks

================================================================================
=== Instance: /FIFO_top/dut
=== Design Unit: work.FIFO
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        27        27         0   100.00%

================================Branch Details================================

Branch Coverage for instance /FIFO_top/dut

    Line         Item                     Count     Source
    ----         ----                     -----     ------
  File FIFO.sv
------------------------------------IF Branch------------------------------------
    4                                     2338     Count coming in to IF
    4               1                       74        if (!f_if.rst_n) begin

    9               1                      918        else if (f_if.wr_en && f_if.count <
f_if.FIFO_DEPTH) begin

    15              1                     1346        else begin

Branch totals: 3 hits of 3 branches = 100.00%

------------------------------------IF Branch------------------------------------
    17                                    1346     Count coming in to IF
    17              1                      674                if (f_if.full && f_if.wr_en) //use " &&
" (bitwise and) instead of " & "-----------> Warning: this statement is not a bitwise operation

    19              1                      672                else

Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    25                                    2338     Count coming in to IF
    25              1                       74        if (!f_if.rst_n) begin
```

Code coverage

```
    29              1                        674        else if (f_if.rd_en && f_if.count != 0) begin

    34              1                       1590        else begin

Branch totals: 3 hits of 3 branches = 100.00%


-------------------------------------IF Branch-------------------------------------
    35                                      1590     Count coming in to IF
    35              1                         13             if (f_if.empty && f_if.rd_en )begin

    38              1                       1577             else begin

Branch totals: 2 hits of 2 branches = 100.00%


-------------------------------------IF Branch-------------------------------------
    45                                      2338     Count coming in to IF
    45              1                         74        if (!f_if.rst_n) begin

    48              1                       2264        else begin

Branch totals: 2 hits of 2 branches = 100.00%


-------------------------------------IF Branch-------------------------------------
    50                                      2264     Count coming in to IF
    50              1                        656                if      ( ({f_if.wr_en, f_if.rd_en} ==
2'b10) && !f_if.full)

    53              1                        198                else if ( ({f_if.wr_en, f_if.rd_en} ==
2'b01) && !f_if.empty)

    56              1                        485                else if ({f_if.wr_en, f_if.rd_en} ==
2'b11) begin //------------------------------------->added

                                             925     All False Count

Branch totals: 4 hits of 4 branches = 100.00%


-------------------------------------IF Branch-------------------------------------
    57                                       485     Count coming in to IF
    57              1                        223                        if(f_if.full)begin

    60              1                          9                        else if (f_if.empty) begin

                                             253     All False Count

Branch totals: 3 hits of 3 branches = 100.00%


-------------------------------------IF Branch-------------------------------------
    67                                      1123     Count coming in to IF
    67              1                        330     assign f_if.full = (f_if.count ==
f_if.FIFO_DEPTH)? 1 : 0;
```

```
    67              2                        793     assign f_if.full = (f_if.count ==
f_if.FIFO_DEPTH)? 1 : 0;

Branch totals: 2 hits of 2 branches = 100.00%


-----------------------------------IF Branch------------------------------------

    68                                      1123     Count coming in to IF

    68              1                         48     assign f_if.empty = (f_if.count == 0)? 1 : 0;


    68              2                       1075     assign f_if.empty = (f_if.count == 0)? 1 : 0;

Branch totals: 2 hits of 2 branches = 100.00%


-----------------------------------IF Branch------------------------------------

    70                                      1123     Count coming in to IF

    70              1                         400     assign f_if.almostfull = (f_if.count ==
f_if.FIFO_DEPTH-1)? 1 : 0; //-------------------->fixed from (count == f_if.FIFO_DEPTH-2) to (count ==
f_if.FIFO_DEPTH-1)


    70              2                         723     assign f_if.almostfull = (f_if.count ==
f_if.FIFO_DEPTH-1)? 1 : 0; //-------------------->fixed from (count == f_if.FIFO_DEPTH-2) to (count ==
f_if.FIFO_DEPTH-1)

Branch totals: 2 hits of 2 branches = 100.00%


-----------------------------------IF Branch------------------------------------

    71                                      1123     Count coming in to IF

    71              1                         52     assign f_if.almostempty = (f_if.count == 1)? 1 :
0;


    71              2                       1071     assign f_if.almostempty = (f_if.count == 1)? 1 :
0;

Branch totals: 2 hits of 2 branches = 100.00%



Condition Coverage:

    Enabled Coverage              Bins   Covered    Misses  Coverage

    ----------------              ----      ----    ------  --------

    Conditions                      20        20         0   100.00%


================================Condition Details================================


Condition Coverage for instance /FIFO_top/dut --


  File FIFO.sv

----------------Focused Condition View-------------------

Line       9 Item    1  (f_if.wr_en && (f_if.count < f_if.FIFO_DEPTH))

Condition totals: 2 of 2 input terms covered = 100.00%
```

```
                    Input Term   Covered  Reason for no coverage   Hint
                   -----------  --------  ----------------------  --------------
                    f_if.wr_en         Y
  (f_if.count < f_if.FIFO_DEPTH)         Y

     Rows:       Hits  FEC Target                       Non-masking condition(s)
 ---------  ---------  --------------------             -------------------------
  Row   1:          1  f_if.wr_en_0                     -
  Row   2:          1  f_if.wr_en_1                     (f_if.count < f_if.FIFO_DEPTH)
  Row   3:          1  (f_if.count < f_if.FIFO_DEPTH)_0  f_if.wr_en
  Row   4:          1  (f_if.count < f_if.FIFO_DEPTH)_1  f_if.wr_en

----------------Focused Condition View-------------------
Line       17 Item    1  (f_if.full && f_if.wr_en)
Condition totals: 2 of 2 input terms covered = 100.00%

  Input Term   Covered  Reason for no coverage   Hint
 -----------  --------  ----------------------  --------------
   f_if.full         Y
  f_if.wr_en         Y

     Rows:       Hits  FEC Target            Non-masking condition(s)
 ---------  ---------  --------------------  -------------------------
  Row   1:          1  f_if.full_0           -
  Row   2:          1  f_if.full_1           f_if.wr_en
  Row   3:          1  f_if.wr_en_0          f_if.full
  Row   4:          1  f_if.wr_en_1          f_if.full

----------------Focused Condition View-------------------
Line       29 Item    1  (f_if.rd_en && (f_if.count != 0))
Condition totals: 2 of 2 input terms covered = 100.00%

         Input Term   Covered  Reason for no coverage   Hint
        -----------  --------  ----------------------  --------------
         f_if.rd_en         Y
  (f_if.count != 0)         Y

     Rows:       Hits  FEC Target            Non-masking condition(s)
 ---------  ---------  --------------------  -------------------------
  Row   1:          1  f_if.rd_en_0          -
  Row   2:          1  f_if.rd_en_1          (f_if.count != 0)
  Row   3:          1  (f_if.count != 0)_0   f_if.rd_en
  Row   4:          1  (f_if.count != 0)_1   f_if.rd_en
```

```
----------------Focused Condition View-------------------
Line       35 Item    1  (f_if.empty && f_if.rd_en)
Condition totals: 2 of 2 input terms covered = 100.00%

  Input Term   Covered  Reason for no coverage   Hint
 -----------  --------  ----------------------  --------------
  f_if.empty         Y
  f_if.rd_en         Y

     Rows:       Hits  FEC Target            Non-masking condition(s)
 ---------  ---------  --------------------  -------------------------
  Row   1:          1  f_if.empty_0          -
  Row   2:          1  f_if.empty_1          f_if.rd_en
  Row   3:          1  f_if.rd_en_0          f_if.empty
  Row   4:          1  f_if.rd_en_1          f_if.empty

----------------Focused Condition View-------------------
Line       50 Item    1  ((~f_if.rd_en && f_if.wr_en) && ~f_if.full)
Condition totals: 3 of 3 input terms covered = 100.00%

  Input Term   Covered  Reason for no coverage   Hint
 -----------  --------  ----------------------  --------------
  f_if.rd_en         Y
  f_if.wr_en         Y
   f_if.full         Y

     Rows:       Hits  FEC Target            Non-masking condition(s)
 ---------  ---------  --------------------  -------------------------
  Row   1:          1  f_if.rd_en_0          (~f_if.full && f_if.wr_en)
  Row   2:          1  f_if.rd_en_1          -
  Row   3:          1  f_if.wr_en_0          ~f_if.rd_en
  Row   4:          1  f_if.wr_en_1          (~f_if.full && ~f_if.rd_en)
  Row   5:          1  f_if.full_0           (~f_if.rd_en && f_if.wr_en)
  Row   6:          1  f_if.full_1           (~f_if.rd_en && f_if.wr_en)

----------------Focused Condition View-------------------
Line       53 Item    1  ((f_if.rd_en && ~f_if.wr_en) && ~f_if.empty)
Condition totals: 3 of 3 input terms covered = 100.00%

  Input Term   Covered  Reason for no coverage   Hint
 -----------  --------  ----------------------  --------------
  f_if.rd_en         Y
```

```
  f_if.wr_en         Y
  f_if.empty         Y

     Rows:       Hits  FEC Target            Non-masking condition(s)
 ---------  ---------  --------------------  -------------------------
  Row   1:          1  f_if.rd_en_0          -
  Row   2:          1  f_if.rd_en_1          (~f_if.empty && ~f_if.wr_en)
  Row   3:          1  f_if.wr_en_0          (~f_if.empty && f_if.rd_en)
  Row   4:          1  f_if.wr_en_1          f_if.rd_en
  Row   5:          1  f_if.empty_0          (f_if.rd_en && ~f_if.wr_en)
  Row   6:          1  f_if.empty_1          (f_if.rd_en && ~f_if.wr_en)

----------------Focused Condition View-------------------
Line       56 Item    1  (f_if.rd_en & f_if.wr_en)
Condition totals: 2 of 2 input terms covered = 100.00%

  Input Term   Covered  Reason for no coverage   Hint
 -----------  --------  -----------------------  --------------
  f_if.rd_en         Y
  f_if.wr_en         Y

     Rows:       Hits  FEC Target            Non-masking condition(s)
 ---------  ---------  --------------------  -------------------------
  Row   1:          1  f_if.rd_en_0          f_if.wr_en
  Row   2:          1  f_if.rd_en_1          f_if.wr_en
  Row   3:          1  f_if.wr_en_0          f_if.rd_en
  Row   4:          1  f_if.wr_en_1          f_if.rd_en

----------------Focused Condition View-------------------
Line       67 Item    1  (f_if.count == f_if.FIFO_DEPTH)
Condition totals: 1 of 1 input term covered = 100.00%

                     Input Term   Covered  Reason for no coverage   Hint
                    -----------  --------  -----------------------  --------------
  (f_if.count == f_if.FIFO_DEPTH)         Y

     Rows:       Hits  FEC Target                        Non-masking condition(s)
 ---------  ---------  --------------------              -------------------------
  Row   1:          1  (f_if.count == f_if.FIFO_DEPTH)_0  -
  Row   2:          1  (f_if.count == f_if.FIFO_DEPTH)_1  -

----------------Focused Condition View-------------------
Line       68 Item    1  (f_if.count == 0)
```

```
Condition totals: 1 of 1 input term covered = 100.00%

         Input Term   Covered  Reason for no coverage   Hint
        -----------  --------  ----------------------  --------------
  (f_if.count == 0)         Y

     Rows:       Hits  FEC Target            Non-masking condition(s)
 ---------  ---------  --------------------  -------------------------
  Row   1:          1  (f_if.count == 0)_0   -
  Row   2:          1  (f_if.count == 0)_1   -

----------------Focused Condition View-------------------
Line       70 Item    1  (f_if.count == (f_if.FIFO_DEPTH - 1))
Condition totals: 1 of 1 input term covered = 100.00%

                           Input Term   Covered  Reason for no coverage   Hint
                          -----------  --------  ----------------------  --------------
  (f_if.count == (f_if.FIFO_DEPTH - 1))         Y

     Rows:       Hits  FEC Target                               Non-masking condition(s)
 ---------  ---------  --------------------                     -------------------------
  Row   1:          1  (f_if.count == (f_if.FIFO_DEPTH - 1))_0  -
  Row   2:          1  (f_if.count == (f_if.FIFO_DEPTH - 1))_1  -

----------------Focused Condition View-------------------
Line       71 Item    1  (f_if.count == 1)
Condition totals: 1 of 1 input term covered = 100.00%

         Input Term   Covered  Reason for no coverage   Hint
        -----------  --------  ----------------------  --------------
  (f_if.count == 1)         Y

     Rows:       Hits  FEC Target            Non-masking condition(s)
 ---------  ---------  --------------------  -------------------------
  Row   1:          1  (f_if.count == 1)_0   -
  Row   2:          1  (f_if.count == 1)_1   -


Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      29        29         0   100.00%
```

```
================================Statement Details================================

Statement Coverage for instance /FIFO_top/dut --

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File FIFO.sv
    1                                                module FIFO(FIFO_if f_if);
    2
    3               1                       2338     always_ff @(posedge f_if.clk or negedge
f_if.rst_n) begin
    4                                                   if (!f_if.rst_n) begin
    5               1                         74                f_if.wr_ptr <= 0;
    6               1                         74                f_if.wr_ack <= 0; //----------------
---------->added
    7               1                         74                f_if.overflow <= 0; //-------------
---------->added
    8                                                   end
    9                                                   else if (f_if.wr_en && f_if.count <
f_if.FIFO_DEPTH) begin
    10              1                        918                f_if.mem[f_if.wr_ptr] <= f_if.data_in;
    11              1                        918                f_if.wr_ack <= 1;
    12              1                        918                f_if.wr_ptr <= f_if.wr_ptr + 1;
    13              1                        918             f_if.overflow<=0;
    14                                                  end
    15                                                  else begin
    16              1                       1346                f_if.wr_ack <= 0;
    17                                                          if (f_if.full && f_if.wr_en) //use " &&
" (bitwise and) instead of " & "-----------> Warning: this statement is not a bitwise operation
    18              1                        674                        f_if.overflow <= 1;
    19                                                          else
    20              1                        672                        f_if.overflow <= 0;
    21                                                  end
    22                                              end
```

```
    23

    24              1                        2338     always_ff @(posedge f_if.clk or negedge
f_if.rst_n) begin

    25                                                   if (!f_if.rst_n) begin

    26              1                          74                f_if.rd_ptr <= 0;

    27              1                          74                f_if.underflow<=0; //----------------
--------------->fixed (handling sequentially)

    28                                                   end

    29                                                   else if (f_if.rd_en && f_if.count != 0) begin

    30              1                         674                f_if.data_out <= f_if.mem[f_if.rd_ptr];

    31              1                         674                f_if.rd_ptr <= f_if.rd_ptr + 1;

    32              1                         674             f_if.underflow<=0; //--------------------
------------>fixed (handling sequentially)

    33                                                   end

    34                                                   else begin

    35                                                        if (f_if.empty && f_if.rd_en )begin

    36              1                          13                 f_if.underflow<=1; //----------------
--------------->fixed (handling sequentially)

    37                                                        end

    38                                                        else begin

    39              1                        1577                 f_if.underflow<=0; //----------------
--------------->fixed (handling sequentially)

    40                                                        end

    41                                                   end

    42                                               end

    43

    44              1                        2338     always_ff @(posedge f_if.clk or negedge
f_if.rst_n) begin

    45                                                   if (!f_if.rst_n) begin

    46              1                          74                f_if.count <= 0;

    47                                                   end

    48                                                   else begin

    49                                                        //write
```

```
    50                                                      if      ( ({f_if.wr_en, f_if.rd_en} ==
2'b10) && !f_if.full)

    51              1                       656                     f_if.count <= f_if.count + 1;

    52                                                      //read

    53                                                      else if ( ({f_if.wr_en, f_if.rd_en} ==
2'b01) && !f_if.empty)

    54              1                       198                     f_if.count <= f_if.count - 1;

    55                                                      //handling write and read together

    56                                                      else if ({f_if.wr_en, f_if.rd_en} ==
2'b11) begin //--------------------------------------->added

    57                                                              if(f_if.full)begin

    58              1                       223                             f_if.count <=
f_if.count - 1;

    59                                                              end

    60                                                              else if (f_if.empty) begin

    61              1                         9                             f_if.count <=
f_if.count + 1;

    62                                                              end

    63                                                      end

    64                                                 end

    65                                             end

    66

    67              1                       1124    assign f_if.full = (f_if.count ==
f_if.FIFO_DEPTH)? 1 : 0;

    68              1                       1124    assign f_if.empty = (f_if.count == 0)? 1 : 0;

    69                                              //assign underflow = (f_if.empty && f_if.rd_en)?
1 : 0;   //--------------------------------->fixed (handling sequentially)

    70              1                       1124    assign f_if.almostfull = (f_if.count ==
f_if.FIFO_DEPTH-1)? 1 : 0; //-------------------->fixed from (count == f_if.FIFO_DEPTH-2) to (count ==
f_if.FIFO_DEPTH-1)

    71              1                       1124    assign f_if.almostempty = (f_if.count == 1)? 1 :
0;
```

```
================================================================================

=== Instance: /FIFO_top

=== Design Unit: work.FIFO_top
```

```
================================================================================
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       6         6         0   100.00%

================================Statement Details================================

Statement Coverage for instance /FIFO_top --

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File FIFO_top.sv
    6                                                module FIFO_top();
    7                                                    bit clk;
    8                                                    initial begin
    9               1                           1            clk=0;
    10              1                           1            forever#5 clk=~clk;
    10              2                        4603
    10              3                        4602
    11                                                   end
    12
    13                                                   FIFO_if f_if(clk);
    14                                                   FIFO dut(f_if);
    15
    16                                                   bind FIFO FIFO_sva sva(f_if);
    17
    18                                                   // run test using run_test task
    19                                                   initial begin
    20              1                           1            uvm_config_db #(virtual
FIFO_if)::set(null,"uvm_test_top","FIFO_if",f_if);
    21              1                           1            run_test("FIFO_test");

Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
```

```
    Toggles                          2         2         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /FIFO_top --

                                              Node      1H->0L      0L->1H  "Coverage"
                                              ----------------------------------------
                                               clk           1           1      100.00

Total Node Count     =          1
Toggled Node Count   =          1
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (2 of 2 bins)

============================================================================
=== Instance: /FIFO_wr_rd_seq_pkg
=== Design Unit: work.FIFO_wr_rd_seq_pkg
============================================================================

Assertion Coverage:
    Assertions                       3         3         0   100.00%
----------------------------------------------------------------------------------------------------------------
-------------------------
Name                 File(Line)                   Failure      Pass     Vacuous     Disable     Attempt
Active Peak Active ATV
                                                  Count        Count    Count       Count       Count
Count  Count
----------------------------------------------------------------------------------------------------------------
-------------------------
/FIFO_wr_rd_seq_pkg/FIFO_wr_rd_seq1/body/#ublk#53369879#18/immed__22
                     FIFO_wr_rd_seq.sv(22)            0       1000          0           0        1000
0           0 off
/FIFO_wr_rd_seq_pkg/FIFO_wr_rd_seq1/body/#ublk#53369879#25/immed__29
                     FIFO_wr_rd_seq.sv(29)            0       1000          0           0        1000
0           0 off
/FIFO_wr_rd_seq_pkg/FIFO_wr_rd_seq2/body/#ublk#53369879#46/immed__50
                     FIFO_wr_rd_seq.sv(50)            0        100          0           0         100
0           0 off

============================================================================
=== Instance: /FIFO_rd_seq_pkg
=== Design Unit: work.FIFO_rd_seq_pkg
============================================================================

Assertion Coverage:
```

```
    Assertions                      1         1         0   100.00%

------------------------------------------------------------------------------------------------------------------------
-------------------------
Name                 File(Line)                    Failure      Pass     Vacuous    Disable    Attempt
Active Peak Active ATV
                                                   Count        Count    Count      Count      Count
Count  Count
------------------------------------------------------------------------------------------------------------------------
-------------------------
/FIFO_rd_seq_pkg/FIFO_rd_seq/body/#ublk#43099047#17/immed__21
                     FIFO_rd_seq.sv(21)                0        100          0          0        100
0           0 off


========================================================================================

=== Instance: /FIFO_wr_seq_pkg

=== Design Unit: work.FIFO_wr_seq_pkg

========================================================================================


Assertion Coverage:
    Assertions                      1         1         0   100.00%

------------------------------------------------------------------------------------------------------------------------
-------------------------
Name                 File(Line)                    Failure      Pass     Vacuous    Disable    Attempt
Active Peak Active ATV
                                                   Count        Count    Count      Count      Count
Count  Count
------------------------------------------------------------------------------------------------------------------------
-------------------------
/FIFO_wr_seq_pkg/FIFO_wr_seq/body/#ublk#43108775#17/immed__21
                     FIFO_wr_seq.sv(21)                0        100          0          0        100
0           0 off


========================================================================================

=== Instance: /FIFO_coverage_pkg

=== Design Unit: work.FIFO_coverage_pkg

========================================================================================


Covergroup Coverage:
    Covergroups                     2        na        na   100.00%
        Coverpoints/Crosses        17        na        na        na
            Covergroup Bins        72        72         0   100.00%
------------------------------------------------------------------------------------------------------------------------
---
Covergroup                                          Metric       Goal       Bins    Status


------------------------------------------------------------------------------------------------------------------------
---
 TYPE /FIFO_coverage_pkg/FIFO_coverage/cg_FIFO      100.00%        100          -    Covered
    covered/total bins:                                 68         68          -
```

Functional coverage

missing/total bins:	0	68	-	
% Hit:	100.00%	100	-	
Coverpoint c_wr_en	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	672	1	-	Covered
bin auto[1]	1592	1	-	Covered
Coverpoint c_rd_en	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	1577	1	-	Covered
bin auto[1]	687	1	-	Covered
Coverpoint c_full	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	1294	1	-	Covered
bin auto[1]	970	1	-	Covered
Coverpoint c_almostfull	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	1621	1	-	Covered
bin auto[1]	643	1	-	Covered
Coverpoint c_empty	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	2198	1	-	Covered
bin auto[1]	66	1	-	Covered
Coverpoint c_almostempty	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	2174	1	-	Covered
bin auto[1]	90	1	-	Covered
Coverpoint c_overflow	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	1590	1	-	Covered

bin auto[1]	674	1	-	Covered
Coverpoint c_underflow	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	2251	1	-	Covered
bin auto[1]	13	1	-	Covered
Coverpoint c_wr_ack	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	1346	1	-	Covered
bin auto[1]	918	1	-	Covered
Cross cross_full	100.00%	100	-	Covered
covered/total bins:	8	8	-	
missing/total bins:	0	8	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	223	1	-	Covered
bin <auto[0],auto[1],auto[1]>	91	1	-	Covered
bin <auto[1],auto[0],auto[1]>	451	1	-	Covered
bin <auto[0],auto[0],auto[1]>	205	1	-	Covered
bin <auto[1],auto[1],auto[0]>	262	1	-	Covered
bin <auto[0],auto[1],auto[0]>	111	1	-	Covered
bin <auto[1],auto[0],auto[0]>	656	1	-	Covered
bin <auto[0],auto[0],auto[0]>	265	1	-	Covered
Cross cross_almost_full	100.00%	100	-	Covered
covered/total bins:	8	8	-	
missing/total bins:	0	8	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	125	1	-	Covered
bin <auto[0],auto[1],auto[1]>	59	1	-	Covered
bin <auto[1],auto[0],auto[1]>	330	1	-	Covered
bin <auto[0],auto[0],auto[1]>	129	1	-	Covered
bin <auto[1],auto[1],auto[0]>	360	1	-	Covered
bin <auto[0],auto[1],auto[0]>	143	1	-	Covered
bin <auto[1],auto[0],auto[0]>	777	1	-	Covered
bin <auto[0],auto[0],auto[0]>	341	1	-	Covered
Cross cross_empty	100.00%	100	-	Covered
covered/total bins:	8	8	-	
missing/total bins:	0	8	-	
% Hit:	100.00%	100	-	

Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	9	1	-	Covered
bin <auto[0],auto[1],auto[1]>	4	1	-	Covered
bin <auto[1],auto[0],auto[1]>	38	1	-	Covered
bin <auto[0],auto[0],auto[1]>	15	1	-	Covered
bin <auto[1],auto[1],auto[0]>	476	1	-	Covered
bin <auto[0],auto[1],auto[0]>	198	1	-	Covered
bin <auto[1],auto[0],auto[0]>	1069	1	-	Covered
bin <auto[0],auto[0],auto[0]>	455	1	-	Covered
Cross cross_almost_empty	100.00%	100	-	Covered
covered/total bins:	8	8	-	
missing/total bins:	0	8	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	21	1	-	Covered
bin <auto[0],auto[1],auto[1]>	11	1	-	Covered
bin <auto[1],auto[0],auto[1]>	40	1	-	Covered
bin <auto[0],auto[0],auto[1]>	18	1	-	Covered
bin <auto[1],auto[1],auto[0]>	464	1	-	Covered
bin <auto[0],auto[1],auto[0]>	191	1	-	Covered
bin <auto[1],auto[0],auto[0]>	1067	1	-	Covered
bin <auto[0],auto[0],auto[0]>	452	1	-	Covered
Cross cross_overflow	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	223	1	-	Covered
bin <auto[1],auto[0],auto[1]>	451	1	-	Covered
bin <auto[1],auto[1],auto[0]>	262	1	-	Covered
bin <auto[0],auto[1],auto[0]>	202	1	-	Covered
bin <auto[1],auto[0],auto[0]>	656	1	-	Covered
bin <auto[0],auto[0],auto[0]>	470	1	-	Covered
Illegal and Ignore Bins:				
ignore_bin no_wr_overflow	0		-	ZERO
Cross cross_underflow	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	9	1	-	Covered
bin <auto[1],auto[1],auto[0]>	476	1	-	Covered
bin <auto[0],auto[1],auto[1]>	4	1	-	Covered

```
            bin <auto[0],auto[1],auto[0]>                198          1          -    Covered
            bin <auto[1],auto[0],auto[0]>                1107         1          -    Covered
            bin <auto[0],auto[0],auto[0]>                470          1          -    Covered
        Illegal and Ignore Bins:
            ignore_bin no_rd_underflow                   0                       -    ZERO
    Cross cross_wr_ack                                 100.00%        100        -    Covered
        covered/total bins:                                6          6          -
        missing/total bins:                                0          6          -
        % Hit:                                         100.00%        100        -
        Auto, Default and User Defined Bins:
            bin <auto[1],auto[1],auto[1]>                262          1          -    Covered
            bin <auto[1],auto[0],auto[1]>                656          1          -    Covered
            bin <auto[1],auto[1],auto[0]>                223          1          -    Covered
            bin <auto[0],auto[1],auto[0]>                202          1          -    Covered
            bin <auto[1],auto[0],auto[0]>                451          1          -    Covered
            bin <auto[0],auto[0],auto[0]>                470          1          -    Covered
        Illegal and Ignore Bins:
            ignore_bin no_wr_ack                         0                       -    ZERO
 TYPE /FIFO_coverage_pkg/FIFO_coverage/cg_rst          100.00%        100        -    Covered
    covered/total bins:                                    4          4          -
    missing/total bins:                                    0          4          -
    % Hit:                                             100.00%        100        -
    Coverpoint c_rst_n                                 100.00%        100        -    Covered
        covered/total bins:                                4          4          -
        missing/total bins:                                0          4          -
        % Hit:                                         100.00%        100        -
 Covergroup instance \/FIFO_coverage_pkg::FIFO_coverage::cg_rst
                                                       100.00%        100        -    Covered
    covered/total bins:                                    4          4          -
    missing/total bins:                                    0          4          -
    % Hit:                                             100.00%        100        -
    Coverpoint c_rst_n                                 100.00%        100        -    Covered
        covered/total bins:                                4          4          -
        missing/total bins:                                0          4          -
        % Hit:                                         100.00%        100        -
        bin asserted                                      37          1          -    Covered
        bin deasserted                                  2264          1          -    Covered
        bin assert_edge                                   36          1          -    Covered
        bin deassert_edge                                 36          1          -    Covered


COVERGROUP COVERAGE:

----------------------------------------------------------------------------------------------------
---
Covergroup                                             Metric       Goal       Bins    Status
```

```
--------------------------------------------------------------------------------------------------------
---
 TYPE /FIFO_coverage_pkg/FIFO_coverage/cg_FIFO        100.00%        100          -    Covered
    covered/total bins:                                    68         68          -
    missing/total bins:                                     0         68          -
    % Hit:                                            100.00%        100          -
    Coverpoint c_wr_en                                100.00%        100          -    Covered
        covered/total bins:                                 2          2          -
        missing/total bins:                                 0          2          -
        % Hit:                                        100.00%        100          -
        bin auto[0]                                       672          1          -    Covered
        bin auto[1]                                      1592          1          -    Covered
    Coverpoint c_rd_en                                100.00%        100          -    Covered
        covered/total bins:                                 2          2          -
        missing/total bins:                                 0          2          -
        % Hit:                                        100.00%        100          -
        bin auto[0]                                      1577          1          -    Covered
        bin auto[1]                                       687          1          -    Covered
    Coverpoint c_full                                 100.00%        100          -    Covered
        covered/total bins:                                 2          2          -
        missing/total bins:                                 0          2          -
        % Hit:                                        100.00%        100          -
        bin auto[0]                                      1294          1          -    Covered
        bin auto[1]                                       970          1          -    Covered
    Coverpoint c_almostfull                           100.00%        100          -    Covered
        covered/total bins:                                 2          2          -
        missing/total bins:                                 0          2          -
        % Hit:                                        100.00%        100          -
        bin auto[0]                                      1621          1          -    Covered
        bin auto[1]                                       643          1          -    Covered
    Coverpoint c_empty                                100.00%        100          -    Covered
        covered/total bins:                                 2          2          -
        missing/total bins:                                 0          2          -
        % Hit:                                        100.00%        100          -
        bin auto[0]                                      2198          1          -    Covered
        bin auto[1]                                        66          1          -    Covered
    Coverpoint c_almostempty                          100.00%        100          -    Covered
        covered/total bins:                                 2          2          -
        missing/total bins:                                 0          2          -
        % Hit:                                        100.00%        100          -
        bin auto[0]                                      2174          1          -    Covered
        bin auto[1]                                        90          1          -    Covered
    Coverpoint c_overflow                             100.00%        100          -    Covered
```

covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	1590	1	-	Covered
bin auto[1]	674	1	-	Covered
Coverpoint c_underflow	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	2251	1	-	Covered
bin auto[1]	13	1	-	Covered
Coverpoint c_wr_ack	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	1346	1	-	Covered
bin auto[1]	918	1	-	Covered
Cross cross_full	100.00%	100	-	Covered
covered/total bins:	8	8	-	
missing/total bins:	0	8	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	223	1	-	Covered
bin <auto[0],auto[1],auto[1]>	91	1	-	Covered
bin <auto[1],auto[0],auto[1]>	451	1	-	Covered
bin <auto[0],auto[0],auto[1]>	205	1	-	Covered
bin <auto[1],auto[1],auto[0]>	262	1	-	Covered
bin <auto[0],auto[1],auto[0]>	111	1	-	Covered
bin <auto[1],auto[0],auto[0]>	656	1	-	Covered
bin <auto[0],auto[0],auto[0]>	265	1	-	Covered
Cross cross_almost_full	100.00%	100	-	Covered
covered/total bins:	8	8	-	
missing/total bins:	0	8	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	125	1	-	Covered
bin <auto[0],auto[1],auto[1]>	59	1	-	Covered
bin <auto[1],auto[0],auto[1]>	330	1	-	Covered
bin <auto[0],auto[0],auto[1]>	129	1	-	Covered
bin <auto[1],auto[1],auto[0]>	360	1	-	Covered
bin <auto[0],auto[1],auto[0]>	143	1	-	Covered
bin <auto[1],auto[0],auto[0]>	777	1	-	Covered
bin <auto[0],auto[0],auto[0]>	341	1	-	Covered

Cross cross_empty	100.00%	100	-	Covered
covered/total bins:	8	8	-	
missing/total bins:	0	8	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	9	1	-	Covered
bin <auto[0],auto[1],auto[1]>	4	1	-	Covered
bin <auto[1],auto[0],auto[1]>	38	1	-	Covered
bin <auto[0],auto[0],auto[1]>	15	1	-	Covered
bin <auto[1],auto[1],auto[0]>	476	1	-	Covered
bin <auto[0],auto[1],auto[0]>	198	1	-	Covered
bin <auto[1],auto[0],auto[0]>	1069	1	-	Covered
bin <auto[0],auto[0],auto[0]>	455	1	-	Covered
Cross cross_almost_empty	100.00%	100	-	Covered
covered/total bins:	8	8	-	
missing/total bins:	0	8	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	21	1	-	Covered
bin <auto[0],auto[1],auto[1]>	11	1	-	Covered
bin <auto[1],auto[0],auto[1]>	40	1	-	Covered
bin <auto[0],auto[0],auto[1]>	18	1	-	Covered
bin <auto[1],auto[1],auto[0]>	464	1	-	Covered
bin <auto[0],auto[1],auto[0]>	191	1	-	Covered
bin <auto[1],auto[0],auto[0]>	1067	1	-	Covered
bin <auto[0],auto[0],auto[0]>	452	1	-	Covered
Cross cross_overflow	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	223	1	-	Covered
bin <auto[1],auto[0],auto[1]>	451	1	-	Covered
bin <auto[1],auto[1],auto[0]>	262	1	-	Covered
bin <auto[0],auto[1],auto[0]>	202	1	-	Covered
bin <auto[1],auto[0],auto[0]>	656	1	-	Covered
bin <auto[0],auto[0],auto[0]>	470	1	-	Covered
Illegal and Ignore Bins:				
ignore_bin no_wr_overflow	0		-	ZERO
Cross cross_underflow	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	

```
        Auto, Default and User Defined Bins:
            bin <auto[1],auto[1],auto[1]>                    9          1          -    Covered
            bin <auto[1],auto[1],auto[0]>                  476          1          -    Covered
            bin <auto[0],auto[1],auto[1]>                    4          1          -    Covered
            bin <auto[0],auto[1],auto[0]>                  198          1          -    Covered
            bin <auto[1],auto[0],auto[0]>                 1107          1          -    Covered
            bin <auto[0],auto[0],auto[0]>                  470          1          -    Covered
        Illegal and Ignore Bins:
            ignore_bin no_rd_underflow                       0                     -    ZERO
    Cross cross_wr_ack                                 100.00%        100          -    Covered
        covered/total bins:                                  6          6          -
        missing/total bins:                                  0          6          -
        % Hit:                                         100.00%        100          -
        Auto, Default and User Defined Bins:
            bin <auto[1],auto[1],auto[1]>                  262          1          -    Covered
            bin <auto[1],auto[0],auto[1]>                  656          1          -    Covered
            bin <auto[1],auto[1],auto[0]>                  223          1          -    Covered
            bin <auto[0],auto[1],auto[0]>                  202          1          -    Covered
            bin <auto[1],auto[0],auto[0]>                  451          1          -    Covered
            bin <auto[0],auto[0],auto[0]>                  470          1          -    Covered
        Illegal and Ignore Bins:
            ignore_bin no_wr_ack                             0                     -    ZERO
TYPE /FIFO_coverage_pkg/FIFO_coverage/cg_rst           100.00%        100          -    Covered
    covered/total bins:                                      4          4          -
    missing/total bins:                                      0          4          -
    % Hit:                                             100.00%        100          -
    Coverpoint c_rst_n                                 100.00%        100          -    Covered
        covered/total bins:                                  4          4          -
        missing/total bins:                                  0          4          -
        % Hit:                                         100.00%        100          -
Covergroup instance \/FIFO_coverage_pkg::FIFO_coverage::cg_rst
                                                       100.00%        100          -    Covered
    covered/total bins:                                      4          4          -
    missing/total bins:                                      0          4          -
    % Hit:                                             100.00%        100          -
    Coverpoint c_rst_n                                 100.00%        100          -    Covered
        covered/total bins:                                  4          4          -
        missing/total bins:                                  0          4          -
        % Hit:                                         100.00%        100          -
        bin asserted                                        37          1          -    Covered
        bin deasserted                                    2264          1          -    Covered
        bin assert_edge                                     36          1          -    Covered
        bin deassert_edge                                   36          1          -    Covered
```

```
TOTAL COVERGROUP COVERAGE: 100.00%  COVERGROUP TYPES: 2


DIRECTIVE COVERAGE:
--------------------------------------------------------------------------------------------
Name                                     Design Design   Lang File(Line)      Hits Status
                                         Unit   UnitType
--------------------------------------------------------------------------------------------
/FIFO_top/dut/sva/cover__threshold       FIFO_sva Verilog  SVA  FIFO_sva.sv(123)2264 Covered
/FIFO_top/dut/sva/cover__rd_wraparound   FIFO_sva Verilog  SVA  FIFO_sva.sv(118)  68 Covered
/FIFO_top/dut/sva/cover__wr_wraparound   FIFO_sva Verilog  SVA  FIFO_sva.sv(113)  97 Covered
/FIFO_top/dut/sva/cover__underflow_detection
                                         FIFO_sva Verilog  SVA  FIFO_sva.sv(108)  13 Covered
/FIFO_top/dut/sva/cover__overflow_detection
                                         FIFO_sva Verilog  SVA  FIFO_sva.sv(103) 663 Covered
/FIFO_top/dut/sva/cover__write_ack       FIFO_sva Verilog  SVA  FIFO_sva.sv(98)  903 Covered
/FIFO_top/dut/sva/async_and_comb_checks/cover_reset_flags
                                         FIFO_sva Verilog  SVA  FIFO_sva.sv(19)   37 Covered
/FIFO_top/dut/sva/async_and_comb_checks/cover_RST_wr_ptr
                                         FIFO_sva Verilog  SVA  FIFO_sva.sv(23)   37 Covered
/FIFO_top/dut/sva/async_and_comb_checks/cover_RST_rd_ptr
                                         FIFO_sva Verilog  SVA  FIFO_sva.sv(27)   37 Covered
/FIFO_top/dut/sva/async_and_comb_checks/cover_RST_count
                                         FIFO_sva Verilog  SVA  FIFO_sva.sv(31)   37 Covered
/FIFO_top/dut/sva/async_and_comb_checks/cover_async_reset_seq
                                         FIFO_sva Verilog  SVA  FIFO_sva.sv(35)   37 Covered
/FIFO_top/dut/sva/async_and_comb_checks/cover_empty_flag
                                         FIFO_sva Verilog  SVA  FIFO_sva.sv(42)   88 Covered
/FIFO_top/dut/sva/async_and_comb_checks/cover_full_flag
                                         FIFO_sva Verilog  SVA  FIFO_sva.sv(48)  737 Covered
/FIFO_top/dut/sva/async_and_comb_checks/cover_almost_full
                                         FIFO_sva Verilog  SVA  FIFO_sva.sv(54)  618 Covered
/FIFO_top/dut/sva/async_and_comb_checks/cover_almost_empty
                                         FIFO_sva Verilog  SVA  FIFO_sva.sv(60)   85 Covered


TOTAL DIRECTIVE COVERAGE: 100.00%  COVERS: 15


ASSERTION RESULTS:
-------------------------------------------------------------------------------
Name                 File(Line)                   Failure      Pass     Vacuous    Disable    Attempt
  Active Peak Active ATV
                                                  Count        Count    Count      Count      Count
  Count  Count
-------------------------------------------------------------------------------
```

```
/FIFO_top/dut/sva/async_and_comb_checks/assert_reset_flags

                    FIFO_sva.sv(17)                    0          37           0           0          37
       0           0 off

/FIFO_top/dut/sva/async_and_comb_checks/assert_RST_wr_ptr

                    FIFO_sva.sv(21)                    0          37           0           0          37
       0           0 off

/FIFO_top/dut/sva/async_and_comb_checks/assert_RST_rd_ptr

                    FIFO_sva.sv(25)                    0          37           0           0          37
       0           0 off

/FIFO_top/dut/sva/async_and_comb_checks/assert_RST_count

                    FIFO_sva.sv(29)                    0          37           0           0          37
       0           0 off

/FIFO_top/dut/sva/async_and_comb_checks/assert_async_reset_seq

                    FIFO_sva.sv(33)                    0          37           0           0          37
       0           0 off

/FIFO_top/dut/sva/async_and_comb_checks/assert_empty_flag

                    FIFO_sva.sv(40)                    0          88           0           0          88
       0           0 off

/FIFO_top/dut/sva/async_and_comb_checks/assert_full_flag

                    FIFO_sva.sv(46)                    0         737           0           0         737
       0           0 off

/FIFO_top/dut/sva/async_and_comb_checks/assert_almost_full

                    FIFO_sva.sv(52)                    0         618           0           0         618
       0           0 off

/FIFO_top/dut/sva/async_and_comb_checks/assert_almost_empty

                    FIFO_sva.sv(58)                    0          85           0           0          85
       0           0 off

/FIFO_top/dut/sva/assert_write_ack

                    FIFO_sva.sv(95)                    0         903        1346          52        2301
       0           2 off

/FIFO_top/dut/sva/assert_overflow_detection

                    FIFO_sva.sv(100)                   0         663        1590          48        2301
       0           2 off

/FIFO_top/dut/sva/assert_underflow_detection

                    FIFO_sva.sv(105)                   0          13        2251          37        2301
       0           2 off

/FIFO_top/dut/sva/assert_wr_wraparound

                    FIFO_sva.sv(110)                   0          97        2165          39        2301
       0           1 off

/FIFO_top/dut/sva/assert_rd_wraparound

                    FIFO_sva.sv(115)                   0          68        2196          37        2301
       0           1 off

/FIFO_top/dut/sva/assert_threshold

                    FIFO_sva.sv(120)                   0        2264           0          37        2301
       0           1 off

/FIFO_wr_rd_seq_pkg/FIFO_wr_rd_seq1/body/#ublk#53369879#18/immed__22

                    FIFO_wr_rd_seq.sv(22)              0        1000           0           0        1000
       0           0 off

/FIFO_wr_rd_seq_pkg/FIFO_wr_rd_seq1/body/#ublk#53369879#25/immed__29

                    FIFO_wr_rd_seq.sv(29)              0        1000           0           0        1000
       0           0 off

/FIFO_wr_rd_seq_pkg/FIFO_wr_rd_seq2/body/#ublk#53369879#46/immed__50
```

```
                    FIFO_wr_rd_seq.sv(50)              0        100          0          0        100
       0           0 off

/FIFO_rd_seq_pkg/FIFO_rd_seq/body/#ublk#43099047#17/immed__21

                    FIFO_rd_seq.sv(21)                 0        100          0          0        100
       0           0 off

/FIFO_wr_seq_pkg/FIFO_wr_seq/body/#ublk#43108775#17/immed__21

                    FIFO_wr_seq.sv(21)                 0        100          0          0        100
       0           0 off


Total Coverage By Instance (filtered view): 100.00%
```